IES ALBARREGAS

TRABAJO DE FIN DE CICLO

CURSO ACADÉMICO 2025/2026

CONVOCATORIA JUNIO

CENTRO EDUCATIVO: IES ALBARREGAS

Manual del Programador

SWAPLT

Alumno: Borja Delgado Sánchez

DNI: 54332162-K

Ciclo: Desarrollo de Aplicaciones Web

Tutor del curso: FRANCISCO JOSE MERA CALDERON

Tutor del proyecto: FRANCISCO JOSE MERA CALDERON

Fecha de entrega: x/06/2026

Índice

1. Introducción **3**
1.1. Nombre del Proyecto 3
1.2. Propósito del Proyecto 3
1.3. Tecnologías Utilizadas 4
1.4. Alcance del Manual 5
2. Arquitectura del Sistema **6**
2.1. Visión General de la Arquitectura 6
2.2. Componentes Principales 8
2.3. Diagramas 10
2.4. Estructura del Proyecto (Directorios) 13
3. Configuracion del Entorno de Desarrollo **18**
3.1. Requisitos Previos 18
3.2. Instalación de Dependencias 20
3.3. Configuración de Variables de Entorno 22
3.4. Ejecución del Proyecto 26
3.5. Herramientas de Desarrollo Adicionales 27
4. Diseño Detallado (Backend) **29**
4.1. Endpoints de la API RESTful 29
4.2. Lógica de Negocio Principal 36
4.3. Acceso a Datos (ORM, Relaciones y Queries Importantes) 40
4.4. Manejo de Errores en el Backend 44
5. Diseño Detallado (Frontend) **49**
5.1. Componentes Reutilizables 49
5.2. Gestión del Estado 51
5.3. Enrutamiento 56
5.4. Interacción con la API del Backend 61
6. Base de Datos **67**
6.1. Esquema de la Base de Datos 67
6.2. Relaciones Importantes en la Base de Datos 70
6.3. Migraciones de Base de Datos 70
7. Pruebas **76**
7.1. Estrategia General de Pruebas 76
7.2. Pruebas del Backend (Laravel) 76

7.3. Pruebas del Frontend (Angular) 78
7.4. Pruebas Manuales de API 80

8. Contribución y Mantenimiento 83
8.1. Guia de Estilo de Codigo 83
8.2. Flujo de Trabajo con Git 84
9. Glosario 85
9.1. Terminología Específica de SWAPLT 86
9.2. Conceptos Técnicos Fundamentales 87
9.3. Patrones de Diseño Aplicados 89
9.4. Acrónimos y Abreviaturas Comunes 92
9.5. Términos Clave del Dominio de SWAPLT 93

1. Introducción

1.1. Nombre del Proyecto

SWAPLT - Compra Venta

1.2. Propósito del Proyecto

SWAPLT es una plataforma nacional diseñada para facilitar la compraventa de vehículos de segunda mano directamente entre particulares, eliminando la necesidad de intermediarios. El proyecto surge como respuesta a la problemática actual del mercado de segunda mano, donde los intermediarios suelen adquirir vehículos a precios bajos para revenderlos con márgenes elevados, lo que reduce el control de los usuarios particulares sobre el valor de sus vehículos y limita las opciones de transacción directa y justa.

El objetivo principal de SWAPLT es devolver el control a los usuarios, ofreciendo un proceso de compraventa transparente, ágil y sin comisiones, al tiempo que se garantiza la seguridad y una comunicación clara entre las partes involucradas.

Los usuarios objetivo de la plataforma son:

- **Particulares que desean vender su vehículo directamente sin recurrir a concesionarios o casas de compraventa.**
- **Compradores particulares que buscan adquirir vehículos de manera directa y sin comisiones.**
- **Pequeños negocios o talleres locales que quieran publicar vehículos reacondicionados o de ocasión sin tener que depender de grandes plataformas.**

1.3. Tecnologías Utilizadas

El desarrollo de SWAPLT se ha realizado utilizando las siguientes tecnologías y herramientas principales:

- **Backend:**
  - **Framework: Laravel v12**
  - **Lenguaje: PHP 8+**
  - **Autenticación: JWT (JSON Web Tokens) para la API.**
  - **ORM: Eloquent**
  - **Entorno de ejecucion: Railway (cloud)**
- **Frontend:**
  - **Framework: Angular v14**
  - **Lenguaje: TypeScript**
  - **Hosting: Vercel (CDN GOLBAL)**
  - **Consumo de API: Módulo HttpClient de Angular.**
- **Base de Datos:**
  - **Gestor: MySQL**
  - **RailWay (servicios gestionados)**
- **Autenticación de Usuarios (Login):**
  - **Inicio de sesión a través de cuentas de Google (implica OAuth).**
  - **Gestion de sesiones mediante el token JWT**
- **APIs Externas:**
  - **Google Maps API: Para la geolocalización de vehículos.**
- **Infraestructura y Despliegue:**
  - **Plataforma Backend: Railway**
  - **Plataforma Frontend: Vercel**
  - **Integración continua: GitHub (CI/CD automático)**
  - **Dominio: Nominalia (swaplt-cars.es)**
- **Herramientas de Desarrollo:**
  - **Editor de Código: Cursor (basado en Visual Studio y Antigravity).**
  - **Control de Versiones: Git y GitHub.**

- **Pruebas de API: Postman.**
- **Navegador para Pruebas: Google Chrome Developer Tools.**

1.4. Alcance del Manual

Este manual del programador está dirigido a:

- Cualquier desarrollador que necesite entender la estructura interna de la aplicación SWAPLT para continuar con su desarrollo, realizar modificaciones o realizar tareas de mantenimiento.

- El autor del proyecto (Borja Delgado Sánchez), como referencia futura para comprender la implementación del sistema.

Se asume que el lector posee conocimientos básicos de desarrollo web y está familiarizado con las tecnologías utilizadas.

Este documento proporciona información detallada sobre:

- **La arquitectura general del sistema SWAPLT.**
- **La configuración del entorno de desarrollo.**
- **La estructura y organización del backend (Laravel) y frontend (Angular).**
- **Los principales endpoints de la API RESTful.**
- **El diseño de la base de datos MySQL.**
- **El proceso de despliegue en un entorno cloud utilizando Railway y Vercel.**
- **La comunicación entre los distintos componentes del sistema.**

La arquitectura de SWAPLT define la estructura y organización de sus componentes tecnológicos, así como la interacción entre ellos para ofrecer las

funcionalidades de la plataforma de forma eficiente y escalable.

---

2. Arquitectura del Sistema

2.1. Visión General de la Arquitectura

SWAPLT adopta una arquitectura Cliente-Servidor desacoplada. Esta se compone de dos bloques principales e independientes que colaboran para ofrecer la experiencia completa al usuario:

- **El Cliente (Frontend): Desarrollado con el framework Angular (versión 14) y TypeScript. Esta es la aplicación con la que el usuario interactúa directamente a través de su navegador web. Se encarga de toda la presentación visual, la interfaz de usuario y la captura de las acciones del usuario. Por ejemplo, Angular es responsable de mostrar los vehículos de manera atractiva y con un estilo consistente, reutilizando componentes para los diferentes anuncios.**
- **El Servidor (Backend): Implementado con el framework Laravel (versión 12) y el lenguaje PHP 8+. Este componente actúa como el "cerebro" de la aplicación, gestionando la lógica de negocio, el acceso a los datos y la seguridad. No tiene una interfaz visual directa para el usuario final, sino que ofrece sus servicios al cliente.**

La comunicación principal entre el cliente (Angular) y el servidor (Laravel) se realiza a través de una API RESTful. El frontend solicita información y acciones al backend mediante peticiones HTTP (por ejemplo, una petición GET a /api/vehiculos/pagina/1 para obtener un listado paginado de los primeros 10 vehículos). El backend procesa estas peticiones, interactúa con la base de datos (MySQL) si es necesario, y devuelve una respuesta al frontend, generalmente en

formato JSON. Esta aproximación, incluyendo la paginación de resultados, está diseñada para optimizar los tiempos de carga y mejorar la experiencia de usuario y la estética de la página.

Internamente, el backend de Laravel está organizado siguiendo el patrón arquitectónico MVC (Modelo-Vista-Controlador).

- **Modelos (Model): Representan los datos de la aplicación y gestionan la interacción con la base de datos MySQL a través del ORM Eloquent.**
- **Controladores (Controller): Manejan las peticiones entrantes de la API desde el cliente Angular, aplican la lógica de negocio necesaria (posiblemente interactuando con los modelos) y preparan la respuesta que se enviará de vuelta.**
- **Vistas (View): En el contexto de SWAPLT, dado que el frontend es gestionado por Angular, el componente "Vista" de Laravel no se utiliza para renderizar HTML directamente para el usuario. En su lugar, las "vistas" del backend se centran en la preparación y serialización de los datos que se enviarán como respuesta de la API (generalmente en formato JSON) al cliente Angular.**

Esta separación clara entre el frontend y el backend permite un desarrollo más modular, facilita el mantenimiento y posibilita que cada parte pueda evolucionar de forma independiente.

2.2. Componentes Principales

La arquitectura Cliente-Servidor de SWAPLT se materializa a través de tres componentes fundamentales que trabajan en conjunto:

1. **Frontend (Cliente Angular): Es la aplicación con la que el usuario interactúa directamente. Desarrollada con Angular v14 y TypeScript, sus responsabilidades clave incluyen:**
    - **Presentación e Interfaz de Usuario (UI): Renderizar toda la información visual de la plataforma, gestionar la navegación entre las distintas secciones y ofrecer una experiencia de usuario intuitiva y responsive, adaptada a ordenadores y dispositivos móviles.**
    - **Interacción con el Usuario: Capturar las entradas del usuario (formularios, clics, búsquedas) y proporcionar feedback visual.**
    - **Comunicación con el Backend: Consumir la API RESTful expuesta por el backend (Laravel) mediante el módulo HttpClient de Angular para solicitar datos (ej. listado de vehículos, detalles de un anuncio) y enviar información (ej. crear un nuevo anuncio, enviar un mensaje).**
    - **Integración con Servicios Externos:**
        - **Gestionar el inicio de sesión de usuarios a través de sus cuentas de Google, interactuando con la API de Google para la autenticación.**
        - **Integrar la API de Google Maps para mostrar la geolocalización de los vehículos anunciados.**
    - **Control de Acceso (Lado Cliente): Gestionar la visibilidad y el acceso a determinadas rutas o funcionalidades basándose en los permisos del usuario autenticado (aunque la validación final de seguridad siempre recae en el backend).**
2. **Backend (Servidor Laravel): Es el núcleo lógico y de procesamiento de SWAPLT, construido con Laravel v12 y PHP 8+. Sus funciones esenciales son:**
    - **Gestión de la API RESTful: Proveer y manejar todos los endpoints de la API que el frontend consume. Esto incluye definir rutas (/routes/api.php), procesar las solicitudes HTTP y generar respuestas (generalmente en JSON).**

- **Lógica de Negocio: Implementar todas las reglas y funcionalidades específicas de SWAPLT, como:**
    - **La gestión del catálogo de vehículos, incluyendo la lógica de paginación y la aplicación de filtros de búsqueda.**
    - **Funcionalidades de interacción entre usuarios, como el sistema de mensajería interna, el bloqueo de usuarios y la gestión de valoraciones (mencionado en entidades de BD).**
    - **Gestión de anuncios, favoritos y estadísticas de visitas a vehículos.**
- **Seguridad y Autenticación:**
    - **Proteger los endpoints de la API mediante autenticación basada en JSON Web Tokens (JWT).**
    - **Gestionar roles y permisos de usuario a través de middleware personalizado.**
    - **Asegurar el cifrado de contraseñas en la base de datos.**
- **Interacción con la Base de Datos: Realizar todas las operaciones de creación, lectura, actualización y borrado (CRUD) sobre la base de datos MySQL. Esto se gestiona a través de los Modelos de Eloquent (/app/Models) y las migraciones (/database/migrations) para la estructura de la base de datos.**
- **Controladores (Controllers): Contienen la lógica para manejar las solicitudes específicas de la API, interactuando con los modelos y otros servicios (/app/Http/Controllers).**

3. **Base de Datos (MySQL): Es el sistema gestor de base de datos relacional encargado del almacenamiento persistente y organizado de toda la información de SWAPLT. Sus principales cometidos son:**
    - **Almacenamiento Seguro de Datos: Guardar de forma fiable la información crítica del sistema, incluyendo datos de usuarios (nombre, correo electrónico, imagen de perfil de Google), detalles de vehículos, anuncios, imágenes, mensajes entre usuarios, favoritos, bloqueos, y otros datos relevantes como visitas o valoraciones.**

- **Integridad y Relaciones: Mantener la coherencia de los datos y gestionar las relaciones entre las diferentes tablas (ej. la relación entre un usuario y sus vehículos, o entre un anuncio y sus mensajes). El esquema de la base de datos se ha diseñado utilizando Mermaid.js (o una herramienta similar) para visualizar estas relaciones.**

Estos tres componentes, aunque desarrollados con tecnologías distintas y con responsabilidades separadas, están intrínsecamente conectados para conformar la aplicación SWAPLT en su totalidad.

2.3. Diagramas

Los diagramas visuales son herramientas fundamentales para comprender la estructura y el funcionamiento de un sistema de software. A continuación, se presentan los diagramas clave que describen la arquitectura de SWAPLT.

1. **Diagrama General de Componentes (Arquitectura Lógica)**

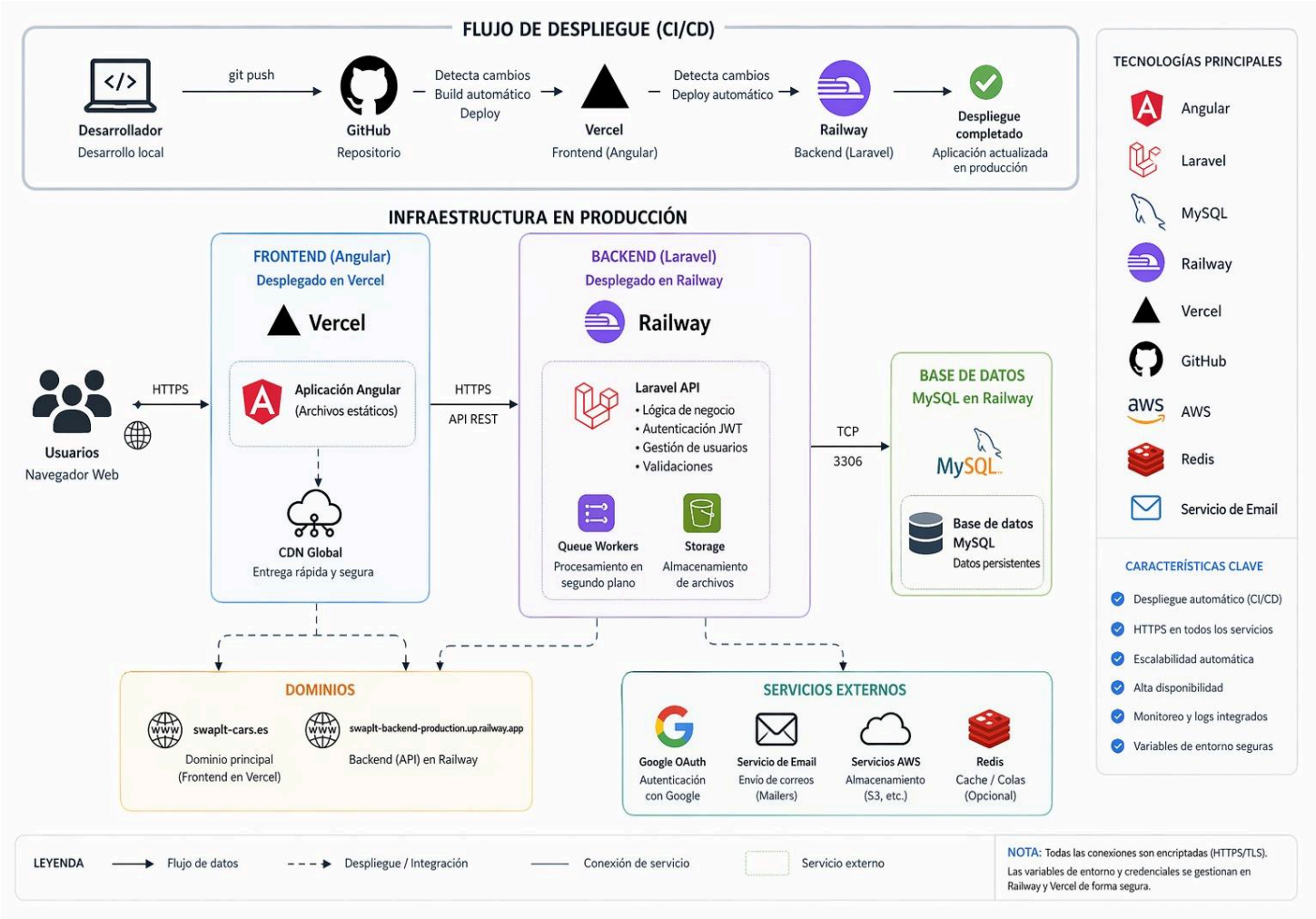
Este diagrama ilustra los principales bloques funcionales de SWAPLT y las interacciones de alto nivel entre ellos. Muestra cómo el frontend, el backend y la base de datos colaboran para ofrecer las funcionalidades de la aplicación.

- **Frontend (Angular): Actúa como la capa de presentación e interacción con el usuario. Se ejecuta en el navegador del cliente y es responsable de realizar peticiones HTTP (principalmente a través de una API RESTful) al Backend para solicitar o enviar información.**
- **API Backend (Laravel): Es el núcleo lógico del sistema. Recibe las peticiones del Frontend, procesa la lógica de negocio correspondiente, interactúa con la Base de Datos para operaciones de persistencia y**

recuperación de datos, y devuelve las respuestas al Frontend (usualmente en formato JSON).

- **Base de Datos (MySQL): Es el sistema de almacenamiento persistente de la aplicación. Contiene toda la información de usuarios, vehículos, anuncios, etc., y es gestionada exclusivamente por el Backend.**

1. **Diagrama de Despliegue**



Nodos de Ejecución y Componentes

Dispositivo del Usuario

Nodo:
Cualquier dispositivo (PC, portátil, tablet o móvil) con acceso a internet.

Componentes residentes:

- **Navegador Web Moderno (ej: Chrome, Firefox, Edge, Safari): entorno de ejecución del cliente.**
- **Aplicación Frontend SWAPLT (Angular en ejecución): el código Angular (HTML, CSS y JavaScript) se descarga desde el servidor y se ejecuta en el navegador, gestionando la interfaz de usuario y la interacción con el sistema.**

Plataforma Frontend (Vercel)

Nodo:
Infraestructura cloud gestionada por Vercel.

Componentes:

- **Aplicación Frontend SWAPLT (Angular compilado):**
  Archivos estáticos optimizados generados tras el proceso de build (`dist/`).
- **CDN (Content Delivery Network):**
  Distribuye los archivos del frontend globalmente para mejorar el rendimiento y reducir la latencia.
- **Dominio personalizado:**
  `swaplt-cars.es`, configurado para apuntar al despliegue en Vercel.

Plataforma Backend (Railway)

Nodo:
Infraestructura cloud gestionada por Railway.

Componentes:

- **Backend SWAPLT (API Laravel, PHP):**
  Aplicación Laravel que expone una API RESTful, encargada de procesar la lógica de negocio, autenticación (JWT) y gestión de datos.
- **Base de Datos SWAPLT (MySQL):**
  Instancia de base de datos gestionada en Railway, donde se almacenan los datos persistentes de la aplicación.
- **Sistema de colas (Queue):**
  Permite la ejecución de tareas en segundo plano (por ejemplo, envíos de correo o procesos diferidos).

Servicios Externos

Nodo:
Infraestructura de servicios externos.

Componentes:

- **API Google Identity (Google Sign-In):**
  Servicio de autenticación que permite a los usuarios iniciar sesión mediante su cuenta de Google.
- **API Google Maps:**
  Utilizada para la visualización de mapas y geolocalización de vehículos.

Comunicación entre Componentes

- **El usuario accede a la aplicación mediante el dominio `swaplt-cars.es`.**

- **El frontend es servido por Vercel y ejecutado en el navegador.**
- **El frontend realiza peticiones HTTP al backend en Railway:**

  https://swaplt-backend-production.up.railway.app/api

- **El backend procesa las peticiones y accede a la base de datos MySQL.**
- **Los servicios externos (Google) se utilizan para autenticación y funcionalidades adicionales.**
  - ■

2.4. Estructura del Proyecto (Directorios)

A continuación, se describe la organización de los directorios principales tanto para el proyecto backend (Laravel) como para el frontend (Angular). Esta estructura ayuda a localizar el código y entender la separación de responsabilidades dentro de la aplicación SWAPLT.

1. **BACKEND**

La estructura del backend sigue en gran medida las convenciones de Laravel, organizando el código de manera lógica.

Directorios Principales dentro de app/:

- **app/Http/Controllers/: Aquí reside la lógica que maneja las peticiones HTTP entrantes desde el frontend.**
  - **AuthController.php: Gestiona todos los aspectos de la autenticación del sistema, incluyendo el login y registro estándar.**
  - **GoogleAuthController.php: Maneja específicamente la integración con la autenticación de Google.**

    - **VehiculoController.php: Es uno de los controladores más extensos y se encarga de todas las operaciones CRUD (Crear, Leer, Actualizar, Borrar) para los vehículos, así como la lógica de listados y filtros.**
    - **UserController.php: Administra los perfiles de usuario y otras operaciones relacionadas con los usuarios.**
    - **VehiculoImagenController.php: Controla la subida, gestión y eliminación de imágenes asociadas a los vehículos.**
    - **MensajeController.php: Gestiona el sistema de mensajería entre usuarios.**
    - **ValoracionController.php: Maneja la lógica para el sistema de valoraciones.**
    - **VehiculoReporteController.php: Administra la creación y gestión de reportes sobre vehículos.**
- **app/Http/Middleware/: Contiene clases que filtran las peticiones HTTP. En SWAPLT se utilizan para la gestión de CORS, la autenticación (asegurar que el usuario esté logueado) y la verificación de roles de usuario, añadiendo capas de seguridad.**
- **app/Models/: Define las entidades principales del sistema a través de modelos de Eloquent, que interactúan con la base de datos.**
    - **User.php: Modelo central para la gestión de usuarios y sus datos.**
    - **Vehiculo.php: Modelo clave que gestiona toda la información de los vehículos en la plataforma.**
    - **VehiculoImagen.php: Modelo para las imágenes asociadas a los vehículos.**
    - **Mensaje.php: Representa los mensajes del sistema de chat entre usuarios.**
    - **Valoracion.php: Modelo para el sistema de valoraciones.**
    - **Categoria.php: Define las categorías para la clasificación de vehículos.**
    - **Favorito.php: Gestiona los vehículos que los usuarios marcan como favoritos.**

    - **VehiculoReporte.php: Modelo para el sistema de reportes de vehículos.**
    - **UserBlock.php: Representa los bloqueos entre usuarios.**
- **app/Providers/: Contiene la configuración de los proveedores de servicios de Laravel, esenciales para el arranque y funcionamiento de la aplicación.**
- **app/Console/: Utilizado para definir comandos personalizados de Artisan, si se necesitasen para tareas programadas o de mantenimiento.**

Otros Directorios y Archivos Clave:

- **database/migrations/: Contiene las migraciones de la base de datos, que permiten versionar y modificar el esquema de la base de datos de forma programática. (Información del PDF)**
- **database/seeders/ y database/factories/: Utilizados para poblar la base de datos con datos de prueba (seeders) o para generar datos falsos para desarrollo y pruebas (factories). (Información del PDF)**
- **routes/api.php: Define todas las rutas para la API RESTful que consume el frontend. Aquí se asocian las URLs de la API con los métodos de los controladores. (Información del PDF)**
- **.env: Archivo de configuración del entorno. Almacena variables sensibles y específicas del entorno, como credenciales de base de datos, claves de API, etc. (Información del PDF)**

Aspectos Destacables de la Estructura Backend: El backend de SWAPLT presenta una clara separación de responsabilidades, con una fuerte integración de servicios de autenticación (incluyendo Google) y sistemas robustos para la gestión de vehículos (imágenes, reportes, favoritos), mensajería, valoraciones y moderación (bloqueos, reportes).

1. **FRONTEND**

La estructura del proyecto frontend en Angular está diseñada para ser modular y escalable, facilitando la organización y el mantenimiento del código.

Directorios Principales dentro de src/app/:

- **/components: Contiene componentes de interfaz de usuario reutilizables que se utilizan en múltiples partes de la aplicación. Esto promueve la consistencia y reduce la duplicación de código.**
    - **Ejemplos identificados: language-selector (selector de idioma), google-ads (componente para integración con Google Ads).**
- **/pages: Agrupa los componentes que representan páginas completas o vistas principales de la aplicación. Cada página puede estar compuesta por varios componentes más pequeños.**
    - **Incluye funcionalidades como autenticación, gestión de vehículos, perfil de usuario, catálogo y mensajería.**
- **/services: Contiene los servicios de Angular. Estos son responsables de la lógica de negocio que no es visual (como la comunicación con el backend API), el manejo del estado y el compartir datos entre componentes.**
    - **Servicios clave identificados:**
        - **auth.service.ts: Maneja la autenticación y autorización de usuarios.**
        - **translation.service.ts: Gestiona la internacionalización y los textos en diferentes idiomas.**
        - **favoritos.service.ts: Encargado de la funcionalidad de vehículos favoritos.**
        - **mensaje.service.ts: Gestiona los mensajes del sistema de chat.**
        - **theme.service.ts: Maneja la temática visual de la aplicación.**
- **/shared: Almacena elementos que se comparten a lo largo de toda la aplicación, como interfaces TypeScript, tipos de datos, constantes, funciones de utilidad y posiblemente componentes o módulos muy genéricos.**
- **/admin: Contiene componentes, servicios y módulos específicos para la sección de administración de SWAPLT, como paneles de control y herramientas de gestión.**

- **/interceptors: Alberga interceptores HTTP. Estos se utilizan para interceptar las peticiones HTTP salientes y las respuestas entrantes, permitiendo realizar tareas como añadir tokens de autenticación a las cabeceras, manejar errores de forma global o transformar datos.**
- **/models: Define las interfaces y tipos de TypeScript que describen la estructura de los datos manejados por la aplicación frontend (ej. la forma de un objeto Vehículo o Usuario que se recibe de la API).**
- **/guards: Contiene los "route guards" de Angular. Estos se utilizan para proteger rutas, decidiendo si un usuario puede acceder a una página específica basándose en si está autenticado o si tiene los permisos necesarios.**
- **/directives: Aquí se colocan directivas personalizadas de Angular, que permiten extender o modificar el comportamiento de los elementos del DOM.**
- **/src/environments/: Contiene archivos de configuración específicos para los diferentes entornos (ej. desarrollo, producción), como las URLs de la API backend. (Información del PDF)**

Aspectos Destacables de la Estructura Frontend: La estructura del frontend sigue un patrón modular y bien organizado, separando claramente las responsabilidades. Esto facilita el desarrollo, las pruebas y el mantenimiento de la aplicación.

---

3. Configuracion del Entorno de Desarrollo

Esta sección describe los pasos y herramientas necesarias para configurar un entorno de desarrollo local que permita trabajar en el proyecto SWAPLT, tanto en su backend (Laravel) como en su frontend (Angular).

3.1. Requisitos Previos

Antes de comenzar con la configuración del proyecto SWAPLT, es fundamental asegurarse de que el siguiente software y herramientas estén instalados y configurados correctamente en el sistema del desarrollador:

- **Sistema Operativo:**
  - **El desarrollo original se realizó en Windows 11. Sin embargo, dado que las tecnologías utilizadas son multiplataforma, el proyecto debería ser compatible también con distribuciones de Linux (como Ubuntu) o macOS.**
- **Para el Desarrollo Backend (Laravel):**
  - **PHP: Versión 8.2.**
  - **Composer: Versión 2.8.6 (o la más reciente compatible). Composer es el gestor de dependencias para PHP y es esencial para instalar las librerías de Laravel.**
  - **Servidor de Base de Datos MySQL: Se requiere una instancia de MySQL para el desarrollo local. Se puede utilizar un gestor visual como MySQL Workbench (como hizo el desarrollador original) o instalar directamente el servidor MySQL.**

- **Para el Desarrollo Frontend (Angular):**
  - **Node.js: Se recomienda una versión LTS (Long Term Support) compatible con Angular 14.0.6 (por ejemplo, Node.js v16.x o v18.x). Node.js incluye NPM.**

    - **NPM (Node Package Manager): Se instala automáticamente con Node.js y es necesario para gestionar las dependencias del frontend y ejecutar comandos de Angular.**
- **Herramientas Generales de Desarrollo:**
    - **Git: Para el control de versiones y la clonación del repositorio del proyecto.**
    - **Editor de Código: Se recomienda un editor de código moderno que soporte PHP y TypeScript/Angular. El desarrollo original se realizó con Cursor, pero otras opciones populares como Visual Studio Code (VS Code) son igualmente válidas.**
    - **Navegador Web Moderno: Para probar la aplicación frontend (ej. Google Chrome, Firefox, Edge).**

Nota sobre Docker: Docker se utiliza para el entorno de desarrollo local de SWAPLT .

Asegurarse de tener estos requisitos cubiertos facilitará el proceso de instalación y configuración del proyecto que se describirá en los siguientes apartados.

3.2. Instalación de Dependencias

Una vez que se han cumplido los requisitos previos y se ha clonado el código fuente del proyecto SWAPLT en el entorno de desarrollo local, el siguiente paso es instalar todas las librerías y paquetes necesarios para que tanto el backend como el frontend funcionen correctamente.

Nota Preliminar: Se asume que el código fuente del proyecto ya ha sido clonado en la máquina del desarrollador. Si el backend y el frontend residen en el mismo repositorio en carpetas separadas (ej: swaplt-laravel/ y swaplt-angular/) o en repositorios separados, el desarrollador deberá navegar (usando el comando cd) al directorio raíz de cada parte (backend y frontend) para ejecutar los comandos de instalación de dependencias correspondientes.

- **Dependencias del BackEnd (Laravel)**

Siga estos pasos desde la terminal, ubicándose en el directorio raíz del proyecto backend

Instalar dependencias de PHP con Composer: Este comando lee el archivo composer.json y descarga todas las librerías PHP necesarias para Laravel y el proyecto SWAPLT.

composer install

Configurar el archivo de entorno: Copie el archivo de ejemplo .env.example a un nuevo archivo llamado .env. Este archivo contendrá las variables de entorno específicas para su configuración local.

cp .env.example .env

> ***Posteriormente, deberá revisar y actualizar el archivo .env con la configuración de su base de datos local, claves de API (si son necesarias para desarrollo) y otras configuraciones del entorno.***

Generar la clave de la aplicación Laravel: Esta clave es utilizada por Laravel para el cifrado y es fundamental para la seguridad de la aplicación.

php artisan key:generate

Ejecutar las migraciones de la base de datos: Este comando creará la estructura de tablas en su base de datos basándose en los archivos de migración del proyecto.

php artisan migrate

> ***Si desea poblar la base de datos con datos de prueba (seeders), puede ejecutar php artisan db:seed después de las migraciones, si los seeders están configurados.***

- **Dependencias del FrontEnd (Angular)**

Siga estos pasos desde la terminal, ubicándose en el directorio raíz del proyecto frontend

Instalar Angular CLI globalmente (Recomendado, si no se tiene): Si no tiene instalada la versión específica de Angular CLI utilizada en el proyecto, es recomendable instalarla globalmente. La versión utilizada es 14.2.13.

npm install -g @angular/cli@14.2.6

> ***Este paso solo es necesario una vez por máquina de desarrollo, o si necesita cambiar entre versiones de Angular CLI.***

Instalar dependencias del proyecto Angular: Este comando lee el archivo package.json del proyecto frontend y descarga todas las librerías y paquetes de Node.js necesarios para Angular y las funcionalidades de SWAPLT.

npm install

> ***Este comando instalará tanto las dependencies (como @angular/core, bootstrap, ngx-toastr, etc.) como las devDependencies (como***

@angular-devkit/build-angular, jasmine-core, etc.) especificadas en el package.json.

Verificar la instalación (Opcional): Puede verificar que Angular CLI y el proyecto están configurados correctamente con:

ng version

3.3. Configuración de Variables de Entorno

La configuración del entorno es crucial para asegurar que la aplicación SWAPLT funcione correctamente en el entorno de desarrollo local y para proteger información sensible como claves de API.

- **Backend (Laravel)**

El backend de Laravel utiliza un archivo `.env` en la raíz del proyecto para gestionar su configuración de entorno. Este archivo se crea copiando el archivo `.env.example` (como se describió en la sección de instalación de dependencias). A continuación, se detallan las variables más importantes que un desarrollador debe revisar y, posiblemente, ajustar en su archivo `.env` local:

- **Información de la Aplicación:**
  - **APP_NAME: Nombre de la aplicación (ej: "SWAPLT").**
  - **APP_ENV: Entorno de la aplicación (normalmente local para desarrollo).**
  - **APP_KEY: Clave de encriptación de la aplicación (se genera con php artisan key:generate).**
  - **APP_DEBUG: Activa/desactiva el modo debug (normalmente true en local).**

- **APP_URL: URL base de la aplicación backend para desarrollo (ej: http://localhost:8000).**
- **Configuración de Logs:**
  - **LOG_CHANNEL: Dónde se guardan los logs (ej: stack, single).**
  - **LOG_LEVEL: Nivel mínimo de logs a registrar (ej: debug).**
- **Conexión a la Base de Datos: El proyecto utiliza MySQL. Deberás configurar las siguientes variables para tu base de datos local:**
  - **DB_CONNECTION=mysql**
  - **DB_HOST=127.0.0.1 (o la IP de tu servidor de BD si es diferente).**
  - **DB_PORT=3306 (o el puerto de tu servidor de BD).**
  - **DB_DATABASE=swaplt_db (o el nombre de tu base de datos local).**
  - **DB_USERNAME=root (o tu usuario de BD local).**
  - **DB_PASSWORD=123456789 (o tu contraseña de BD local).**

Nota: **Aunque se mencionó que SQLite podría estar configurado por defecto para desarrollo en el PDF, los valores provistos son para MySQL. Si se desea usar SQLite, se deberá cambiar DB_CONNECTION=sqlite y eliminar las otras variables DB_HOST, DB_PORT, DB_USERNAME, DB_PASSWORD. La ruta de la base de datos SQLite se configuraría entonces por defecto en database/database.sqlite.**

- **Sesiones, Caché y Colas:**
  - **SESSION_DRIVER=file**
  - **QUEUE_CONNECTION=database (Esto implica que la tabla de colas debe existir; se crea con php artisan queue:table y php artisan migrate).**
  - **CACHE_STORE=database (Esto implica que la tabla de caché debe existir; se crea con php artisan cache:table y php artisan migrate).**

    - ***Para desarrollo local, si no se quieren usar colas o caché basadas en base de datos, se pueden cambiar a sync para colas y array o file para caché para simplificar.***
- **Configuración de Correo Electrónico (MAIL): Para el envío de correos (ej. verificación de cuenta). Para desarrollo local, es común usar servicios como MailHog o simplemente registrar los correos en un archivo para evitar el envío real.**

    - **MAIL_MAILER: Configurado como smtp. Para desarrollo, se puede cambiar a log.**
    - **MAIL_HOST: smtp.gmail.com (si se usa Gmail y se configura para permitir aplicaciones menos seguras o una contraseña de aplicación).**
    - **MAIL_PORT: 587**
    - **MAIL_USERNAME: swaplt.comprayventa@gmail.com**
    - **MAIL_PASSWORD: (La contraseña de aplicación de Gmail proporcionada).**
    - **MAIL_ENCRYPTION: tls**
    - **MAIL_FROM_ADDRESS: swaplt.comprayventa@gmail.com**
    - **MAIL_FROM_NAME: "SWAPLT"**

- **JWT (JSON Web Tokens):**

    - **JWT_SECRET: Se proporciona un valor de ejemplo. Este secreto es crucial para la firma de los tokens JWT. Se puede generar uno nuevo usando php artisan jwt:secret si es necesario.**
- **Servicios de Google (para Google Sign-In):**

    - **GOOGLE_CLIENT_ID: (El Client ID proporcionado).**
    - **GOOGLE_CLIENT_SECRET: (El Client Secret proporcionado).**

    - **GOOGLE_REDIRECT_URI: http://localhost:8000/api/auth/google/callback (Asegúrate de que esta URI esté registrada en tus credenciales de la Consola de Google Cloud).**
- **URL del Frontend:**
    - **FRONTEND_URL=http://localhost:4200 (Usado, por ejemplo, en redirecciones o construcción de URLs en correos).**
- **Frontend (Angular)**

El frontend de Angular gestiona su configuración de entorno a través de archivos TypeScript ubicados en la carpeta `src/environments/`. Típicamente, encontrarás dos archivos:

- **environment.ts: Usado para el entorno de desarrollo.**
- **environment.prod.ts: Usado para el entorno de producción.**

Cuando ejecutas `ng serve`, se usa `environment.ts` por defecto. Cuando construyes para producción (`ng build --prod`), se usa environment.prod.ts. Deberás revisar y configurar las variables en `src/environments/environment.ts` para tu desarrollo local

- **Las variables clave que has proporcionado para el frontend son:**
    - **production: Debería ser false en environment.ts y true en environment.prod.ts.**
    - **apiUrl: La URL base del backend Laravel. Para desarrollo local, según tu configuración, es 'http://localhost:8000/api'.**
    - **googleMapsApiKey: La clave de API para Google Maps, necesaria para la funcionalidad de geolocalización. (Valor proporcionado: 'AIzaSyCIzQc-xMyVBcN04ZyjRrTSTLQZwVycUe0').**

- **googleTranslateApiKey: Clave de API para Google Translate. (Valor proporcionado: 'AIzaSyAMY8sX9zAq2pKrREO_mByynsbVAsfOaJ0').**

Es importante que estas claves de API (Google Maps, Google Translate) estén correctamente configuradas y tengan los permisos necesarios en la Consola de Google Cloud para el dominio en el que se está desarrollando (ej. localhost debe estar permitido para desarrollo).

3.4. Ejecución del Proyecto

Una vez que el entorno de desarrollo está completamente configurado, con las dependencias instaladas y las variables de entorno ajustadas, puede proceder a ejecutar el backend y el frontend de SWAPLT en su máquina local.

- **Ejecución del Backend (Laravel)**

Navegar al directorio del backend: Abra una terminal o línea de comandos y navegue hasta el directorio raíz del proyecto backend de Laravel (ej: swaplt-laravel/).

Iniciar el servidor de desarrollo de Laravel: Ejecute el siguiente comando Artisan para iniciar el servidor PHP de desarrollo incorporado:

php artisan serve

> **Por defecto, el servidor de desarrollo de Laravel se ejecutará en la dirección http://127.0.0.1:8000 o http://localhost:8000. Verá un mensaje en la terminal confirmando esto. Ahora el backend está escuchando peticiones API.**

- **Ejecución del Frontend (Angular)**

Navegar al directorio del frontend: Abra una nueva terminal o línea de comandos (manteniendo la terminal del backend en ejecución) y navegue hasta el directorio raíz del proyecto frontend de Angular (ej: swaplt-angular/).

Iniciar el servidor de desarrollo de Angular: Ejecute el siguiente comando de Angular CLI para compilar la aplicación y iniciar el servidor de desarrollo:

ng s

> **Por defecto, el servidor de desarrollo de Angular se ejecutará y será accesible en la dirección http://localhost:4200/. La aplicación se recompilará automáticamente si realiza cambios en los archivos fuente. Puede abrir esta URL en su navegador web para ver y interactuar con la aplicación SWAPLT.**

3.5. Herramientas de Desarrollo Adicionales

Además del editor de código y las herramientas de línea de comandos estándar, las siguientes herramientas fueron utilizadas durante el desarrollo de SWAPLT y pueden ser de utilidad para futuros desarrolladores:

- **Postman para Pruebas de API Backend: Se utilizó Postman extensivamente para probar los endpoints de la API RESTful de Laravel. Se ha proporcionado una colección de Postman (TFC.postman_collection.json) que incluye numerosas peticiones preconfiguradas para las diversas funcionalidades del backend (autenticación, gestión de usuarios, vehículos, categorías, favoritos, mensajes, etc.).**
    - **Uso: Para utilizar esta colección, impórtela en su instancia de Postman. La mayoría de las URLs están configuradas para el**

entorno de desarrollo local (ej: http://localhost:8000/api/ o http://127.0.0.1:8000/api/).

- **Autenticación: Muchas peticiones requieren un token de autenticación (Bearer Token). Deberá ejecutar primero las peticiones de login o registro para obtener un token válido y luego configurarlo manualmente en las cabeceras (Authorization) de las peticiones protegidas o utilizar las capacidades de Postman para gestionar variables de entorno y tokens. Los tokens presentes en la colección exportada probablemente hayan expirado.**

- **Google Chrome Developer Tools: Las herramientas de desarrollo integradas en Google Chrome (u otros navegadores modernos) fueron fundamentales para el desarrollo y depuración del frontend Angular.**
  - **Inspección de Elementos y Estilos: Para verificar y ajustar el HTML y CSS.**
  - **Consola JavaScript: Para visualizar logs, errores y ejecutar comandos.**
  - **Pestaña de Red (Network): Para inspeccionar las peticiones a la API, sus cabeceras y respuestas.**
  - **Modo Responsive (Device Mode): Esencial para verificar la adaptabilidad y el diseño responsive de la interfaz de SWAPLT en diferentes tamaños de pantalla (móviles, tablets).**
  - **Pestañas de Application/Storage: Para revisar el almacenamiento local, de sesión, cookies, etc.**
- **Editor de Código Cursor (con IA integrada): El desarrollo principal del proyecto se llevó a cabo utilizando Cursor, un editor de código basado en VS Code que incorpora funcionalidades de asistencia mediante inteligencia artificial. Estas características pueden haber sido utilizadas para la generación de código, explicación, o depuración.**
- **APIs de Google (Maps, Translate, Identity Platform): Durante el desarrollo y prueba del frontend, se interactuó directamente con varias APIs de Google:**
  - **Google Maps API: Para la integración de mapas y geolocalización.**

- **Google Translate API: (Mencionada en la configuración del frontend). Para funcionalidades de traducción.**
- **Google Identity Platform: Para la autenticación mediante cuentas de Google. Un desarrollador que trabaje en estas integraciones necesitará estar familiarizado con la documentación de estas APIs y asegurarse de que las claves de API estén correctamente configuradas en el frontend.**

---

4. Diseño Detallado (Backend)

Esta sección se enfoca en los aspectos internos del componente backend de SWAPLT, desarrollado con Laravel. Se describirá la interfaz de programación de aplicaciones (API), la lógica de negocio principal, el acceso a datos y el manejo de errores.

4.1. Endpoints de la API RESTful

La API RESTful del backend de SWAPLT es el punto de entrada para todas las interacciones desde el cliente (frontend Angular) y otras posibles aplicaciones. Todos los endpoints de la API están prefijados con `/api/`

La autenticación para los endpoints protegidos se gestiona consistentemente mediante Bearer Tokens JWT, los cuales deben ser enviados en la cabecera Authorization de la petición HTTP.

- **Autenticación (AUTH)**
  - **Endpoints relacionados con el registro, inicio de sesión y gestión de la autenticación de usuarios.**

Registro de Usuario

Método: POST
URL: /api/register
Descripción: Crea una nueva cuenta de usuario.
Autenticación Requerida: No.
Cuerpo (Request Body - JSON):

```
{
   "name": "string",
   "email": "string",
   "password": "string",
   "password_confirmation": "string"
}
```

Login de Usuario

Método: POST
URL: /api/login
Descripción: Autentica a un usuario con su correo electrónico y contraseña, y si es exitoso, devuelve un token JWT.
Autenticación Requerida: No.
Cuerpo (Request Body - JSON):

```
{
   "email": "string",
   "password": "string"
}
```

(Nota: La colección de Postman incluye otros endpoints de autenticación como verificación de correo, reseteo de contraseña, login con Google, logout, obtener datos del usuario autenticado (/me) y refrescar token).

- **Gestión de Usuarios**
    - **Endpoints relacionados con el registro, inicio de sesión y gestión de la autenticación de usuarios.**
        - **Obtener Usuario por ID**

Método: GET
URL: /api/users/{id}
Descripción: Obtiene la información detallada de un usuario específico.
Autenticación Requerida: Sí.
Parámetros de Ruta:
id: ID numérico del usuario.

- **Gestión de Vehículos (VEHICULOS)**
    - **Endpoints dedicados a la creación, consulta, actualización y eliminación de vehículos.**
        - **Crear Vehículo**

Método: POST
URL: /api/vehiculos
Descripción: Crea un nuevo anuncio de vehículo, permitiendo la subida de imágenes asociadas.
Cuerpo (Request Body - form-data):
Autenticación Requerida: Sí.

user_id: ID del usuario propietario.
categoria_id: ID de la categoría del vehículo.
marca: string
modelo: string
precio: number
anio: number (año de fabricación).
estado: string (ej: "nuevo", "usado").
transmision: string (ej: "manual", "automática").
tipo_combustible: string (ej: "gasolina", "diésel").
kilometraje: number
fuerza: number (CV o HP).
capacidad_motor: number (ej: 1.9, 2.0).
color: string
ubicacion: string (dirección o ciudad).
matricula: string (opcional).
numero_serie: string (VIN, opcional).
numero_puertas: number
descripcion: string
vehiculo_robado: string ("sí" o "no").
vehiculo_libre_accidentes: string ("sí" o "no").

(Nota: La colección de Postman incluye endpoints para obtener vehículos con paginación, por ID, por usuario, aplicar filtros, búsquedas y gestionar estadísticas de visitas).

- **Gestión de Categorías (CATEGORIAS)**
  - **Endpoints para listar y gestionar las categorías de los vehículos.**
    - **Obtener Todas las Categorías**

****Método**: GET**
****URL**: /api/categorias**
****Descripción**: Devuelve un listado de todas las categorías de vehículos disponibles en la plataforma.**
****Autenticación Requerida**: Sí.**

(Nota: La colección de Postman incluye también endpoints para obtener categoría por ID, crear, actualizar y eliminar categorías).

- **Gestión de Favoritos (FAVORITOS)**
  - **Endpoints para que los usuarios gestionen su lista de vehículos favoritos.**
    - **Añadir Vehículo a Favoritos**

Método: POST**
URL: /api/favoritos**
Descripción: Permite al usuario autenticado añadir un vehículo a su lista personal de favoritos.**
Autenticación Requerida: Sí.**
Cuerpo (Request Body - JSON):**

```
{
  "vehiculo_id": number
}
```

(Nota: La colección de Postman también incluye endpoints para obtener los favoritos del usuario y eliminar un favorito).

- **Mensajería (MENSAJES)**
  - **Endpoints para la comunicación entre usuarios.**
    - **Obtener Conversación entre Dos Usuarios**

Método: GET**
URL: /api/mensajes/{emisor_id}/{receptor_id}**
Descripción: Recupera el historial de mensajes intercambiados entre el usuario emisor y el receptor.**
Parámetros de Ruta:
emisor_id: ID del usuario emisor.
receptor_id: ID del usuario receptor.
Autenticación Requerida: No**

(Nota: La colección de Postman incluye también endpoints para enviar mensajes, marcar como leídos y eliminar mensajes).

- **Gestión de Valoraciones (VALORACIONES)**
  - **Endpoints para que los usuarios puedan valorar a otros usuarios.**

- **Valorar a un Usuario**

Método: POST
URL: /api/valoraciones
Descripción: Permite a un usuario autenticado enviar una valoración (puntuación y comentario) sobre otro usuario.
Autenticación Requerida: Sí.
Cuerpo (Request Body - JSON):

```
{
  "receptor_id": "number", // ID del usuario que recibe la valoración
  "valor": "number",       // Puntuación (ej. 1 a 5)
  "comentario": "string"
}
```

(Nota: La colección de Postman también incluye un endpoint para obtener las valoraciones de un usuario específico).

- **Gestión de Bloqueos de Usuarios (BLOQUEADOS)**
  - **Endpoints para que los usuarios puedan bloquear y desbloquear a otros usuarios.**

    - **Bloquear a un Usuario**

Método: POST
URL: /api/usuarios/{id}/bloquear
Descripción: Permite al usuario autenticado bloquear a otro usuario, impidiendo interacciones.
Autenticación Requerida: Sí.
Parámetros de Ruta:
id: ID del usuario que va a ser bloqueado.
Cuerpo (Request Body - JSON):

```
{
   "razon": "string" // Motivo del bloqueo (opcional)
}
```

4.2. Lógica de Negocio Principal

El backend de SWAPLT implementa varias lógicas de negocio clave para asegurar el correcto funcionamiento de la plataforma y una buena experiencia de usuario. A continuación, se describen dos de las más fundamentales:

- Sistema de Publicación y Gestión de Vehículos

Esta funcionalidad es el núcleo de la plataforma, permitiendo a los usuarios publicar, visualizar y gestionar anuncios de vehículos.

- Funcionalidad General: Proporciona un sistema completo para que los usuarios puedan listar sus vehículos para la venta o intercambio, incluyendo detalles, imágenes y gestión de su visibilidad.
- Pasos y Reglas Principales Implementadas en el Backend
    - Validación de Datos del Vehículo
        - Se realiza una validación exhaustiva de los datos enviados al crear o actualizar un vehículo.
        - Campos obligatorios como marca, modelo, precio, año, estado, etc., son verificados.

        - **Se comprueba la existencia del usuario propietario y la categoría del vehículo en la base de datos.**
        - **Se validan los tipos de datos para cada campo (numéricos, cadenas de texto, etc.).**
    - **Gestión de Visibilidad y Bloqueos:**
        - **Un aspecto importante es el sistema de filtrado de vehículos basado en los bloqueos entre usuarios.**
        - **Los vehículos pertenecientes a un usuario que ha sido bloqueado por el usuario actual no serán visibles para este último.**
        - **De forma recíproca, se excluyen del listado los vehículos de usuarios que han bloqueado al usuario que realiza la consulta. Esta lógica de negocio es fundamental para mantener una experiencia de usuario segura y respetuosa.**
    - **Sistema de Registro de Visitas:**
        - **El backend registra cada vez que se accede a la vista detallada de un vehículo.**
        - **Se almacena el ID del usuario que realiza la visita (si está autenticado), su dirección IP y la fecha de la visita.**
        - **Esta información permite mantener estadísticas de visitas por cada vehículo, funcionalidad que puede ser consultada a través de la API.**
    - **Gestión de Imágenes de Vehículos:**
        - **Se permite la subida de múltiples imágenes para cada anuncio de vehículo.**
        - **Estas imágenes se almacenan físicamente en el sistema de archivos del servidor.**
        - **La API proporciona las URLs para acceder a estas imágenes y también puede generar vistas previas en formato base64 si es necesario.**
- **Modelos y Controladores Principales Involucrados**

    - **VehiculoController.php: Orquesta la mayoría de las operaciones CRUD (Crear, Leer, Actualizar, Borrar) para los vehículos, aplicando las validaciones y lógicas de negocio.**
    - **VehiculoImagenController.php: Se especializa en la gestión de la subida, almacenamiento y eliminación de las imágenes asociadas a los vehículos.**
    - **Vehiculo.php (Modelo Eloquent): Representa la entidad vehículo en la base de datos y encapsula las relaciones y atributos de los vehículos.**
- **Sistema de Mensajería Interna**

Para facilitar la comunicación directa y privada entre compradores y vendedores, SWAPLT incorpora un sistema de mensajería interna.

- **Funcionalidad General: Permite a los usuarios registrados enviarse mensajes directos para negociar, hacer preguntas sobre los vehículos o coordinar encuentros**
- **Pasos y Reglas Principales Implementadas en el Backend:**
    - **Validación de la Comunicación:**
        - **Antes de procesar un mensaje, se verifica la existencia de ambos usuarios (emisor y receptor) en la base de datos.**
        - **Crucialmente, se integra con el sistema de bloqueo de usuarios: si uno de los usuarios ha bloqueado al otro, la comunicación (envío de nuevos mensajes) entre ellos se impide.**
        - **Se valida el contenido del mensaje (ej. para evitar mensajes vacíos).**
    - **Gestión de Conversaciones:**
        - **Los mensajes se organizan y almacenan manteniendo la relación entre el emisor y el receptor, permitiendo reconstruir el historial de una conversación específica.**

        - **Las conversaciones se presentan y ordenan cronológicamente.**
    - **Control de Estado de los Mensajes:**
        - **El sistema incluye una funcionalidad para marcar los mensajes como leídos o no leídos.**
        - **Se permite a los usuarios eliminar mensajes individuales (la lógica de eliminación puede variar, ej. borrado lógico o físico, y si afecta a uno o ambos participantes).**
        - **Se busca mantener la integridad y el contexto de la conversación.**
    - **Seguridad en la Mensajería:**
        - **Se realizan verificaciones constantes del estado de bloqueo entre los usuarios involucrados en una comunicación.**
        - **Se aplican validaciones de permisos para asegurar que un usuario solo pueda acceder a sus propias conversaciones y no a las de otros.**
        - **Se implementan medidas para proteger contra el acceso no autorizado al contenido de los mensajes.**
- **Modelos y Controladores Principales Involucrados:**
    - **MensajeController.php: Gestiona toda la lógica relacionada con el envío, recepción, visualización y eliminación de mensajes.**
    - **Mensaje.php (Modelo Eloquent): Representa un mensaje individual en la base de datos, almacenando su contenido, emisor, receptor, y estado.**
    - **User.php (Modelo Eloquent): Además de los datos del usuario, este modelo es consultado para gestionar y verificar las relaciones de bloqueo entre usuarios, que afectan directamente al sistema de mensajería.**

Estas dos funcionalidades, con sus respectivas lógicas de negocio, son esenciales para la propuesta de valor de SWAPLT, centrada en la interacción directa entre particulares para la compraventa de vehículos.

4.3. Acceso a Datos (ORM, Relaciones y Queries Importantes)

El backend de SWAPLT utiliza Eloquent ORM, la herramienta integrada en Laravel, para interactuar con la base de datos MySQL de una manera intuitiva y orientada a objetos. Esto significa que cada tabla de la base de datos tiene un "Modelo" correspondiente en PHP que se utiliza para interactuar con esa tabla. Las relaciones entre tablas se definen directamente en estos modelos.

- **Relaciones Principales de Eloquent**
  - **Relaciones del Modelo User (Usuario):**
    - **Usuario y Vehículos (Vehiculo):**
      - **Un usuario puede publicar múltiples vehículos: User hasMany Vehiculo.**
      - **Cada vehículo pertenece a un único usuario: Vehiculo belongsTo User.**
    - **Usuario y Mensajes (Mensaje):**
      - **Un usuario puede ser el emisor de muchos mensajes: User hasMany Mensaje (relación definida como mensajesEnviados).**
      - **Un usuario puede ser el receptor de muchos mensajes: User hasMany Mensaje (relación definida como mensajesRecibidos).**
      - **Cada mensaje tiene un emisor y un receptor, ambos referenciando al modelo User: Mensaje belongsTo User (una relación para el emisor y otra para el receptor).**
    - **Usuario y Valoraciones (Valoracion):**
      - **Un usuario puede emitir múltiples valoraciones hacia otros usuarios: User hasMany Valoracion (como valoracionesEnviadas).**

        - **Un usuario puede recibir múltiples valoraciones de otros usuarios: User hasMany Valoracion (como valoracionesRecibidas).**
        - **Cada valoración tiene un emisor y un receptor, ambos usuarios: Valoracion belongsTo User (una relación para el valorador y otra para el valorado).**
    - **Usuario y Bloqueos de Usuario (UserBlock):**
        - **Un usuario puede bloquear a múltiples otros usuarios: User hasMany UserBlock (como bloqueosRealizados, donde el usuario actual es el blocker_id).**
        - **Un usuario puede ser bloqueado por múltiples otros usuarios: User hasMany UserBlock (como bloqueosRecibidos, donde el usuario actual es el blocked_id).**
        - **Esta implementación incluye métodos auxiliares en el modelo User como estaBloqueadoPor(), haBloqueadoA(), usuariosBloqueados(), etc., para facilitar la lógica de negocio relacionada con los bloqueos.**
  - **Relaciones del Modelo Vehiculo (Vehículo):**
    - **Vehículo e Imágenes (VehiculoImagen):**
        - **Un vehículo puede tener asociadas múltiples imágenes: Vehiculo hasMany VehiculoImagen.**
        - **Cada imagen pertenece a un único vehículo: VehiculoImagen belongsTo Vehiculo.**
        - **Las imágenes pueden tener un orden específico (gestionado a través de un campo como imagen_order).**
    - **Vehículo y Categoría (Categoria):**
        - **Cada vehículo pertenece a una categoría específica: Vehiculo belongsTo Categoria.**

        - **Una categoría puede agrupar múltiples vehículos: Categoria hasMany Vehiculo.**
    - **Vehículo y Favoritos (relación con User a través de la tabla favoritos):**
        - **Existe una relación de muchos a muchos entre usuarios y vehículos para la funcionalidad de "favoritos": Vehiculo belongsToMany User (a través de una tabla pivote favoritos). Esto permite que un usuario pueda tener muchos vehículos favoritos y que un vehículo pueda ser favorito para muchos usuarios.**
    - **Vehículo y Visitas (VehiculoVisita):**
        - **Un vehículo puede registrar múltiples visitas: Vehiculo hasMany VehiculoVisita.**
        - **Cada VehiculoVisita registra el ID del usuario visitante (si está autenticado), su dirección IP y la fecha de la visita.**
  - **Queries y Lógicas de Acceso a Datos Destacables**

Además de las relaciones directas, existen ciertas lógicas o queries específicas en los modelos que son importantes para funcionalidades clave:

Estadísticas de Visitas de Vehículos: El modelo Vehiculo incluye un método como getEstadisticasVisitas($dias = 30) que calcula las visitas diarias para un vehículo durante un período determinado. Un ejemplo de la query subyacente sería:

Esta query agrupa y cuenta las visitas por fecha, proporcionando datos para análisis.

```
public function getEstadisticasVisitas($dias = 30)
{
    return $this->visitas() // Asumiendo que 'visitas' es la relación hasMany a VehiculoVisita
    ->selectRaw('DATE(fecha_visita) as fecha, COUNT(*) as total_visitas')
```

```
        ->where('fecha_visita', '>=', now()->subDays($dias))
        ->groupBy('fecha')
        ->orderBy('fecha')
        ->get();
}
```

Sistema de Bloqueos (Consultas Auxiliares): El modelo User contiene métodos que simplifican la obtención de listas de usuarios relacionados con bloqueos

```
public function usuariosBloqueados()
{
    // Devuelve los usuarios que ESTE usuario ha bloqueado
    return User::whereIn('id', $this->bloqueosRealizados()->pluck('blocked_id'));
}

public function usuariosQueMeBloquearon()
{
    // Devuelve los usuarios que HAN BLOQUEADO a este usuario
    return User::whereIn('id', $this->bloqueosRecibidos()->pluck('blocker_id'));
}
```

- **Características Especiales del Acceso a Datos**
    - **Gestión de Imágenes: Se utiliza una tabla específica (vehiculos_imagenes) para las imágenes de los vehículos, permitiendo la gestión de múltiples imágenes por vehículo, el ordenamiento y el almacenamiento de sus rutas y URLs.**
    - **Sistema de Bloqueos Bidireccional: La estructura de la tabla user_blocks y las relaciones en el modelo User permiten una gestión eficiente de los bloqueos, incluyendo la posibilidad de almacenar la razón del bloqueo.**

- **Sistema de Visitas Detallado: El registro de visitas con usuario (si aplica), IP y fecha permite un análisis estadístico útil.**
- **Valoraciones Bidireccionales: El sistema de valoraciones está claramente definido con relaciones que permiten identificar quién valoró a quién, junto con la puntuación y comentarios.**

El uso consistente de Eloquent ORM y la definición clara de estas relaciones y lógicas de acceso a datos facilitan la manipulación de la información, el mantenimiento del código y la implementación de funcionalidades complejas como filtros de búsqueda, gestión de conversaciones, control de visibilidad y análisis estadístico.

4.4. Manejo de Errores en el Backend

Un manejo de errores robusto y consistente es esencial en SWAPLT para proporcionar una API fiable, facilitar la depuración y ofrecer una buena experiencia al desarrollador del frontend.

- **Formato Estándar de Respuestas de Error de la API**

Cuando la API de SWAPLT encuentra un error, devuelve una respuesta JSON estructurada de manera consistente para facilitar su procesamiento por parte del cliente. El formato general es:

```
{
   "success": false, // Siempre false para indicar un error
   "message": "Descripción legible del error",
   "error": "Un código o identificador breve del error (opcional)",
   "details": {} // Un objeto con detalles específicos del error
}
```

Este formato se utiliza para diversos tipos de errores, adaptando el message, error y details según el contexto.

- **Gestión de Tipos Comunes de Errores HTTP**

SWAPLT maneja los errores HTTP estándar de la siguiente manera:

- **Errores de Validación (HTTP 422 Unprocessable Entity):**
  - **Cuando los datos enviados a un endpoint (ej. al crear o actualizar un recurso) no superan las reglas de validación definidas en los controladores de Laravel (usando $request->validate()), la API responde automáticamente con un código de estado 422.**
  - **La respuesta JSON incluye un mensaje general y un objeto details (o errors según la configuración de Laravel) que desglosa los errores por cada campo validado.**

```
// Ejemplo de validación en un controlador:
 $request->validate([
    'user_id' => 'required|exists:users,id',
    'categoria_id' => 'required|exists:categorias,id',
    // ... más validaciones
 ]);
```

- **Errores de "No Encontrado" (HTTP 404 Not Found):**
  - **Si un cliente intenta acceder a un recurso que no existe (ej. un vehículo con un ID incorrecto), la API devuelve un código de estado 404 con un mensaje JSON claro y específico, como "Vehículo no encontrado".**

```
// Ejemplo de respuesta manual para un 404:
 if (!$vehiculo) {
    return response()->json(['message' => 'Vehículo no encontrado'], 404);
```

```
}
```

- **Errores de Autenticación (HTTP 401 Unauthorized) y Autorización (HTTP 403 Forbidden):**
  - **401 (Unauthorized): Se devuelve si el usuario intenta acceder a un recurso protegido sin proporcionar un token JWT válido, o si el token ha expirado o es incorrecto.**
  - **403 (Forbidden): Se devuelve si el usuario está autenticado pero no tiene los permisos necesarios para realizar una acción o acceder a un recurso específico (ej. intentar modificar datos de otro usuario, o interactuar con un usuario bloqueado).**

```
// Ejemplo de respuesta 403 por bloqueo:
  return response()->json(['message' => 'No puedes enviar mensajes a este usuario debido a un bloqueo'], 403);
```

- **Errores del Servidor (HTTP 500 Internal Server Error):**
  - **Para excepciones no capturadas o errores inesperados durante el procesamiento en el servidor, se devuelve un código de estado 500.**
  - **La respuesta JSON intenta proporcionar un mensaje descriptivo del problema, evitando exponer detalles sensibles de la implementación. Estos errores también son registrados para su posterior análisis.**

```
// Ejemplo de captura genérica de excepciones:
 try {
    // Operación que puede fallar
 } catch (\\Exception $e) {
    Log::error('Error en operación X: ' . $e->getMessage()); // Logging del error
    return response()->json([
        'success' => false,
```

```
        'message' => 'Error al realizar la operación: ' . $e->getMessage() // En desarrollo se puede ser más explícito
    ], 500);
}
```

- **Manejo Específico de Errores de Lógica de Negocio**

Además de los errores HTTP estándar, SWAPLT implementa un manejo de errores para situaciones específicas de su lógica de negocio:

- **Validaciones de Negocio Personalizadas: Se realizan comprobaciones que van más allá de la validación de formato de datos, como impedir que un usuario se valore a sí mismo o que valore a usuarios con los que no ha interactuado. Estos casos suelen devolver un error 403 o 422 con mensajes claros.**
- **Errores en la Gestión de Archivos: Se controla la existencia de archivos, como las imágenes de los vehículos, devolviendo un 404 con detalles si un archivo no se encuentra.**
- **Errores de Autenticación con Servicios Externos (ej. Google): Se manejan los errores que pueden surgir durante la comunicación con APIs de terceros, como Google para el inicio de sesión. La respuesta puede variar según el cliente que realiza la petición (API o navegador), pudiendo redirigir al frontend con un parámetro de error.**
- **Logging de Errores**

El registro de errores es una parte fundamental para el monitoreo y la depuración de la aplicación.

- **Configuración: Las variables de entorno LOG_CHANNEL=stack y LOG_LEVEL=debug (definidas en el archivo .env) controlan el comportamiento del logging.**

- **Implementación: Por defecto, los errores y la información de logging se registran en archivos dentro del directorio storage/logs/laravel.log. Los errores críticos incluyen stack traces completos para facilitar el diagnóstico.**
- **Logging Específico: En partes críticas del código, como en la autenticación con Google o en operaciones importantes, se utiliza el facade Log de Laravel (Log::error(...)) para registrar mensajes de error específicos con contexto adicional.**

Este enfoque estructurado para el manejo y logging de errores contribuye a la estabilidad, seguridad y mantenibilidad de la aplicación SWAPLT.

---

5. Diseño Detallado (Frontend)

Esta sección se adentra en la estructura y las características clave de la aplicación frontend de SWAPLT, desarrollada con Angular. Se describirán los

componentes reutilizables, la gestión del estado, el sistema de enrutamiento y cómo el frontend interactúa con la API del backend.

5.1. Componentes Reutilizables

La aplicación SWAPLT fomenta la reutilización de código y la consistencia visual a través de componentes de interfaz de usuario (UI) bien definidos. Estos componentes se encuentran principalmente en dos directorios:

- `/src/app/components/`**: Contiene componentes reutilizables que son partes de UI más generales o específicas de funcionalidades que se repiten en diversas secciones de la aplicación.**
- `/src/app/shared/components/`**: Es una práctica común en Angular para organizar componentes que son ampliamente compartidos y no pertenecen a una característica específica, asegurando que puedan ser fácilmente importados y utilizados donde se necesiten.**

A continuación, se describen algunos de los componentes reutilizables más importantes de SWAPLT:

- **PaginationComponent**
  - **Ubicación:** `/src/app/shared/components/pagination/`
  - **Propósito Principal: Muestra una barra de paginación estándar con controles (anterior, siguiente, números de página) para permitir al usuario navegar a través de conjuntos de resultados paginados (ej. listados de vehículos, usuarios, etc.).**
  - **Entradas Principales (@Input()):**
    - **currentPage: number: El número de la página actualmente visible.**
    - **totalItems: number: El número total de elementos disponibles en todas las páginas.**

        - **itemsPerPage: number: El número de elementos que se muestran por página.**
    - **Salidas Principales (@Output()):**
        - **pageChange: EventEmitter<number>: Emite el nuevo número de página cuando el usuario selecciona una página diferente.**
- **GoogleAdsComponent**
    - **Ubicación: `/src/app/components/google-ads/`**
    - **Propósito Principal: Encapsula la lógica para mostrar anuncios de Google Ads dentro de la aplicación. Se utiliza en secciones estratégicas como el catálogo de vehículos y otras áreas con el objetivo de monetizar la plataforma.**
- **LanguageSelectorComponent**
    - **Ubicación: `/src/app/components/language-selector/`**
    - **Propósito Principal: Proporciona una interfaz (ej. un menú desplegable) que permite a los usuarios cambiar el idioma de la aplicación SWAPLT. Típicamente se ubica en la barra de navegación principal o en el pie de página.**
- **EstadisticasVehiculoComponent**
    - **Ubicación: `/src/app/pages/mis-vehiculos/estadisticas-vehiculo/`**
    - **Propósito Principal: Muestra estadísticas detalladas y gráficos sobre las visitas que ha recibido un vehículo específico. Aunque ubicado en una ruta de página, su diseño modular permite visualizar información clave como el total de visitas y el promedio diario.**
    - **Entradas Principales (@Input()):**
        - **vehiculoId: string (o number): El ID del vehículo para el cual se mostrarán las estadísticas.**
        - **propietarioId: string (o number): El ID del propietario del vehículo (podría usarse para validaciones o contextos adicionales).**
    - **Características: Incluye la representación gráfica de las visitas.**

- **LoadingComponent (Indicador de Carga Genérico)**
    - **Propósito Principal: Proporciona una indicación visual de carga (ej. un spinner, una barra de progreso) que se muestra mientras se obtienen datos del backend o se realizan operaciones asíncronas.**
    - **Ubicación y Uso: Este es un patrón común implementado, posiblemente a través de un componente muy genérico o una directiva, y se reutiliza en múltiples componentes y páginas (como el catálogo, la sección de mensajes, los detalles de un vehículo, etc.) para informar al usuario que una operación está en curso.**
- **ErrorComponent (Manejo de Errores Visual)**
    - **Propósito Principal: Muestra mensajes de error al usuario de una manera consistente cuando ocurren problemas (ej. fallos al cargar datos, errores de la API). A menudo incluye una breve descripción del error y, opcionalmente, una acción para reintentar la operación.**
    - **Ubicación y Uso: Similar al LoadingComponent, este es un patrón de UI reutilizado en diversos puntos de la aplicación para estandarizar la forma en que se comunican los errores.**

5.2. Gestión del Estado

La gestión del estado en la aplicación frontend de SWAPLT se implementa mediante un enfoque basado en servicios de Angular y la librería RxJS, sin recurrir a librerías de gestión de estado más complejas como NgRx, NGXS o Akita. Este enfoque prioriza la simplicidad y mantenibilidad para la escala actual del proyecto.

Los datos y el estado de la aplicación se manejan principalmente a través de:

- **Servicios de Angular: Actúan como la fuente única de verdad (Single Source of Truth) para piezas específicas del estado.**

- **localStorage del navegador: Se utiliza para persistir datos críticos que deben sobrevivir a la recarga de la página, como el token de autenticación, información del usuario y preferencias.**
- **RxJS (BehaviorSubject, Subject, Observables): Se emplea para crear flujos de datos reactivos, permitiendo que los componentes se suscriban a los cambios de estado y reaccionen en consecuencia.**
- **Decoradores @Input() y @Output(): Se utilizan para la comunicación directa entre componentes padre e hijo.**

- **Servicios Clave en la Gestión del Estado**

  Varios servicios son fundamentales para la gestión del estado global y compartido:

  - **AuthService (auth.service.ts):**
    - **Responsabilidad: Maneja todo lo relacionado con el estado de autenticación del usuario.**
    - **Persistencia: Utiliza localStorage para almacenar el token JWT, los datos del perfil del usuario y su rol.**
    - **Funcionalidades Clave:**
      - `login()`**: Autentica al usuario, almacena el token y los datos del usuario.**
      - `getProfile()`**: Obtiene y almacena los datos del perfil del usuario autenticado.**
      - `isAuthenticated()`**: Verifica si el usuario está actualmente autenticado (generalmente comprobando la validez del token).**
      - `getCurrentUser()`**: Proporciona acceso a los datos del usuario actualmente logueado.**

- ThemeService (theme.service.ts):
    - Responsabilidad: Gestiona el tema visual de la aplicación (ej. claro/oscuro).

Reactividad: Utiliza un BehaviorSubject<boolean> para mantener el estado actual del modo oscuro (darkMode) y un Observable (darkMode$) para que los componentes puedan suscribirse a los cambios de tema

```
// Ejemplo de la lógica en ThemeService
private darkMode = new BehaviorSubject<boolean>(false);
darkMode$ = this.darkMode.asObservable();
```

- Persistencia: Guarda la preferencia de tema del usuario en localStorage.

- Funcionalidades Clave: Proporciona métodos para cambiar el tema y aplicar los estilos correspondientes.

- TranslationEventsService (translation-events.service.ts):

    - Responsabilidad: Maneja los eventos relacionados con el cambio de idioma en la aplicación.

Reactividad: Utiliza un Subject<LanguageChangeEvent> para emitir un evento cada vez que el idioma cambia. Los componentes interesados pueden suscribirse a este Subject para actualizar su contenido.

```
// Ejemplo de la lógica en TranslationEventsService
 private languageChangeSubject = new Subject<LanguageChangeEvent>();
 languageChange$ = this.languageChangeSubject.asObservable();
```

- Funcionalidades Clave: Ofrece métodos para emitir el evento de cambio de idioma y para que otros servicios o componentes se suscriban a dichos

cambios.

- **FavoritosService (favoritos.service.ts):**
    - **Responsabilidad: Administra el estado de la lista de vehículos favoritos del usuario.**

Reactividad: Utiliza un BehaviorSubject<Favorito[]> para mantener la lista actual de favoritos y permitir que los componentes se suscriban a sus actualizaciones.

```
// Ejemplo de la lógica en FavoritosService
 private favoritosSubject = new BehaviorSubject<Favorito[]>([]);
 favoritos$ = this.favoritosSubject.asObservable(); // (Asumo que se expone un observable)
```

- **Funcionalidades Clave: Implementa métodos para cargar, añadir, y eliminar vehículos de la lista de favoritos, actualizando el BehaviorSubject correspondientemente.**

- **Comunicación entre Componentes**

  Se emplea una combinación de:

    - **@Input() y @Output(): Para la comunicación jerárquica directa entre componentes padre e hijo.**
    - **Servicios Compartidos: Para compartir estado o lógica entre componentes que no tienen una relación directa padre-hijo. Los componentes inyectan el servicio y acceden a sus Observables o métodos.**

    - **RxJS Observables: Para una comunicación reactiva, permitiendo que los componentes reaccionen a los cambios de estado emitidos por los servicios.**
- **Principales Piezas de Estado Gestionadas**

    - **Estado de Autenticación: Token JWT, datos del usuario (ID, nombre, email, rol), estado de verificación del correo electrónico.**
    - **Configuración del Usuario: Idioma seleccionado, tema visual (claro/oscuro), y otras posibles preferencias del usuario.**
    - **Estado Específico de la Aplicación: Lista de vehículos favoritos del usuario, estado de los mensajes (ej. mensajes no leídos), estado de los vehículos (ej. al filtrar o buscar en el catálogo), estado de los bloqueos de usuario.**
- **Patrones y Enfoques Adoptados**

    - **Servicios Singleton: Los servicios se proveen generalmente en el root (providedIn: 'root'), asegurando que haya una única instancia de cada servicio en toda la aplicación.**
    - **Reactividad con RxJS: Uso extensivo de Observables, Subjects y BehaviorSubjects para manejar flujos de datos asíncronos y cambios de estado.**
    - **Persistencia en localStorage: Para datos críticos que deben mantenerse entre sesiones, como el estado de autenticación y las preferencias del usuario.**
    - **Interceptores HTTP: Se utilizan para tareas relacionadas con el estado, como adjuntar automáticamente el token JWT a las peticiones salientes o manejar errores de autenticación de forma global.**
- **Este enfoque de gestión del estado, aunque no utiliza una librería de estado dedicada, resulta adecuado para la escala y complejidad actual de SWAPLT, ofreciendo un buen balance entre simplicidad, mantenibilidad y**

funcionalidad reactiva.

5.3. Enrutamiento

El sistema de enrutamiento en la aplicación frontend de SWAPLT, desarrollada con Angular, es responsable de gestionar la navegación del usuario entre las diferentes vistas y funcionalidades. Permite asociar URLs específicas del navegador con los componentes de Angular que deben mostrarse.

- **Definición de Rutas**

  Las definiciones de las rutas en SWAPLT están modularizadas para una mejor organización:

  - **app-routing.module.ts: Contiene las rutas principales y de nivel superior de la aplicación.**
  - **Módulos de Características con Enrutamiento Propio: Existen módulos específicos que definen sus propias rutas, tales como:**
    - **admin-routing.module.ts (para la sección de administración).**
    - **catalogo-routing.module.ts (para las rutas del catálogo de vehículos).**
    - **favoritos-routing.module.ts (para las rutas de la sección de favoritos).**
    - **static.module.ts (para las rutas de páginas estáticas como términos y condiciones, política de privacidad, etc.).**
- **Configuración de Rutas Principales**

  El archivo app-routing.module.ts establece la configuración principal de navegación. Algunas de las rutas clave definidas son:

  - **Redirección por Defecto: La ruta raíz ('') redirige a /catalogo.**

- **Rutas Públicas:**
  - `/login`: **Carga el LoginComponent.**
  - `/register`: **Carga el RegisterComponent.**
  - `/vehiculo/:id`: **Carga el DetallesVehiculoComponent para mostrar los detalles de un vehículo específico (utiliza un parámetro de ruta id).**
  - `/perfil-usuario/:id`: **Carga el PerfilUsuarioComponent para mostrar el perfil público de un usuario (utiliza un parámetro de ruta id).**
- **Rutas Protegidas (requieren autenticación):**
  - `/profile`: **Carga el ProfileComponent.**
  - `/vender-vehiculo`: **Carga el VenderVehiculoComponent.**
  - `/mis-vehiculos`: **Carga el MisVehiculosComponent.**
  - `/mensajes`: **Carga el MensajesComponent.**
- **Módulos con Carga Diferida (Lazy Loading):**
  - `/admin`: **Carga el AdminModule.**
  - `/catalogo`: **Carga el CatalogoModule.**
  - `/favoritos`: **Carga el FavoritosModule.**
  - `/static`: **Carga el StaticModule (para páginas estáticas).**
- **Ruta Comodín (Wildcard Route) para 404:**
  - `*`: **Cualquier ruta no definida anteriormente redirige a /catalogo.**

- **Protección de Rutas (Route Guards)**

SWAPLT utiliza guardias de ruta (Route Guards) para proteger el acceso a ciertas secciones de la aplicación basándose en el estado de autenticación del usuario y sus roles. Los principales guardias son:

- **AuthGuard (guards/auth.guard.ts):**

- **Propósito: Asegura que solo los usuarios autenticados puedan acceder a ciertas rutas.**
- **Funcionamiento: Verifica el estado de autenticación a través del AuthService. Si el usuario no está autenticado, es redirigido a la página de /login.**
- **Rutas Protegidas: /profile, /mis-vehiculos, /mensajes, /favoritos, /catalogo (interesantemente, el catálogo también está protegido según el documento, lo cual puede significar que se requiere login para ver el catálogo completo o ciertas acciones dentro de él), y /editar-vehiculo/:id.**

```
// Lógica simplificada del AuthGuard
 export class AuthGuard implements CanActivate {
   constructor(private authService: AuthService, private router: Router) {}

   canActivate(): boolean {
     if (!this.authService.isAuthenticated()) {
       this.router.navigate(['/login']);
       return false;
     }
     return true;
   }
 }
```

- **AdminGuard (definido en admin/auth.guard.ts o similar):**
  - **Propósito: Restringe el acceso a las rutas de la sección de administración (/admin) únicamente a usuarios con el rol de 'admin'.**
  - **Funcionamiento: Obtiene el usuario actual a través del AuthService y verifica su rol. Si el usuario no es 'admin', es redirigido a una página de acceso denegado (ej. /acceso-denegado).**

- Rutas Protegidas: Todas las sub-rutas dentro de /admin, como /admin/usuarios, /admin/vehiculos, /admin/categorias, /admin/bloqueos.

```
// Lógica simplificada del AdminGuard (asumiendo que es para el rol 'admin')
 export class AdminGuard implements CanActivate { // Nombre de clase puede variar
  constructor(private authService: AuthService, private router: Router) {}

  canActivate(): boolean {
    const user = this.authService.getCurrentUser();
    if (user && user.rol && user.rol.toLowerCase() === 'admin') {
      return true;
    } else {
      this.router.navigate(['/acceso-denegado']); // o a otra ruta
      return false;
    }
  }
}
```

- Uso de Parámetros en Rutas

La aplicación utiliza parámetros de ruta para pasar datos necesarios a los componentes, como el ID de un vehículo o de un usuario. Los componentes acceden a estos parámetros a través del servicio ActivatedRoute de Angular.

```
// Ejemplo de obtención de parámetro 'id' en un componente:
 constructor(private route: ActivatedRoute) {
  this.route.paramMap.subscribe(params => {
    const id = params.get('id');
    // Lógica para cargar datos usando el id
  });
```

```
}
```

La navegación programática a rutas con parámetros se realiza utilizando el servicio Router de Angular (ej. this.router.navigate(['/vehiculo', vehiculoId]);).

- Carga Diferida (Lazy Loading)

Para optimizar el rendimiento inicial de la aplicación, SWAPLT implementa la carga diferida (Lazy Loading) para varios módulos. Esto significa que el código de estos módulos solo se carga cuando el usuario navega a una de sus rutas por primera vez. Los módulos configurados para carga diferida incluyen:

- AdminModule (ruta `/admin`)
- CatalogoModule (ruta `/catalogo`)
- FavoritosModule (ruta `/favoritos`)
- StaticModule (ruta `/static`)
- Manejo de Rutas No Encontradas (404)

Si un usuario intenta acceder a una URL que no coincide con ninguna de las rutas definidas, la aplicación lo gestiona mediante una ruta comodín (**) que redirige al usuario a la página del catálogo (`/catalogo`).

```
{ path: '**', redirectTo: '/catalogo' }
```

Esta configuración de enrutamiento proporciona una navegación estructurada, segura y optimizada para la aplicación SWAPLT.

5.4. Interacción con la API del Backend

La comunicación entre el frontend de Angular y la API del backend de Laravel en SWAPLT se gestiona de manera estructurada utilizando servicios de Angular, el módulo HttpClient, interceptores HTTP para la autenticación, y un manejo tipado de datos y errores.

- **Servicios de Comunicación con la API**

La lógica para interactuar con los endpoints de la API del backend está encapsulada en servicios de Angular, ubicados principalmente en el directorio `/src/app/services/`. Estos servicios se pueden categorizar en:

- **Servicios Generales:**
  - **auth.service.ts: Responsable de los procesos de autenticación, como login, registro, y la gestión del token JWT y la información del usuario.**
  - **mensaje.service.ts: Gestiona la lógica para enviar y recibir mensajes entre usuarios.**
  - **translation.service.ts: Maneja las interacciones con la API de Google Translate para las funcionalidades de traducción de la aplicación.**
- **Servicios Específicos por Recurso: Estos servicios agrupan las llamadas a la API relacionadas con un modelo o recurso específico del backend:**
  - **vehiculos/vehiculo.service.ts: Administra las operaciones CRUD (Crear, Leer, Actualizar, Borrar) para los vehículos.**
  - **vehiculos/vehiculo-imagen.service.ts: Encargado de la subida y gestión de las imágenes asociadas a los vehículos.**

- user/usuarios.service.ts y user/user.service.ts: Gestionan las operaciones relacionadas con los usuarios y sus perfiles.

Todos estos servicios utilizan el módulo HttpClient de Angular para realizar las peticiones HTTP (`GET, POST, PUT, DELETE`.) a la API. La URL base de la API (ej. `http://localhost:8000/api`) se obtiene del archivo de configuración de entorno (src/environments/environment.ts).

- Manejo de Tokens JWT con HTTP Interceptor

Para asegurar que las peticiones a los endpoints protegidos de la API incluyan el token de autenticación JWT, SWAPLT utiliza un interceptor HTTP. Este interceptor, AuthInterceptor, se encuentra en `/src/app/interceptors/auth.interceptor.ts.`

- Funcionamiento del AuthInterceptor:
  - Intercepta cada petición HTTP saliente antes de que sea enviada al servidor.
  - Obtiene el token JWT actual del AuthService.
  - Si existe un token, clona la petición original y añade una cabecera Authorization con el valor Bearer {token}.
  - Permite que la petición (original o clonada con la cabecera) continúe hacia el backend.

```
// Ubicación: /src/app/interceptors/auth.interceptor.ts
 import { Injectable } from '@angular/core';
   import { HttpEvent, HttpInterceptor, HttpHandler, HttpRequest } from '@angular/common/http';
 import { Observable } from 'rxjs';
 import { AuthService } from '../services/auth.service'; // Asegúrate de que la ruta sea correcta

 @Injectable()
```

```
export class AuthInterceptor implements HttpInterceptor {
  constructor(private authService: AuthService) {}

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const token = this.authService.getToken(); // Método para obtener el token desde AuthService
    if (token) {
      req = req.clone({
        setHeaders: { Authorization: `Bearer ${token}` }
      });
    }
    return next.handle(req);
  }
}
```

Este mecanismo centraliza la lógica de adjuntar tokens, evitando tener que hacerlo manualmente en cada llamada de servicio.

- Manejo de Respuestas de la API
  - Respuestas Exitosas:
    - Tipado Fuerte: Los datos recibidos del backend se mapean a interfaces y modelos de TypeScript definidos en el directorio /src/app/models/ (ej. Usuario, Vehiculo, Mensaje). Esto proporciona autocompletado, verificación de tipos en tiempo de desarrollo y un código más robusto.

Transformación de Datos: Se utilizan operadores de RxJS como map dentro de los servicios para transformar la respuesta cruda de la API al formato o

estructura de datos que necesita el frontend, o para extraer la información relevante del objeto de respuesta.

```
// Ejemplo en vehiculo.service.ts (adaptado)
getUserVehicles(): Observable<Vehiculo[]> {
  return this.http.get<any>(`${environment.apiUrl}/user/vehiculos`).pipe(
    map(response => {
        if (response.success && response.data) { // Asumiendo una estructura de respuesta { success: boolean, data: any, message?: string }
        return response.data as Vehiculo[];
      } else {
          throw new Error(response.message || 'Error al obtener los vehículos del usuario');
      }
    })
  );
}
```

- ■
- ○ **Manejo de Error en la API**
  - ■ **En Servicios: Los servicios utilizan operadores de RxJS como catchError para capturar errores provenientes de las llamadas HTTP. Estos errores pueden ser registrados en la consola para depuración y luego relanzados para que el componente que llama al servicio pueda manejarlos.**
  - ■ **Notificaciones al Usuario: Se utiliza la librería ngx-toastr (una dependencia del proyecto) para mostrar notificaciones (toasts) amigables al usuario informando sobre errores o resultados de operaciones. Estas notificaciones se suelen invocar desde los componentes que consumen los servicios, después de recibir una respuesta de error.**

- **Errores HTTP Comunes: El frontend está preparado para interpretar los códigos de estado HTTP comunes devueltos por el backend (401/403 para autenticación/autorización, 404 para recurso no encontrado, 422 para errores de validación, 500 para errores del servidor) y actuar en consecuencia, por ejemplo, redirigiendo al login en caso de un 401, o mostrando mensajes específicos.**

- **Ejemplo de Llamada a la API (Crear Vehículo)**

El siguiente es un ejemplo simplificado de un método en vehiculo.service.ts para crear un nuevo vehículo, ilustrando el uso de HttpClient, la apiUrl y el manejo de errores:

```
import { HttpClient, HttpHeaders } from '@angular/common/http';
 import { Observable, throwError } from 'rxjs';
 import { catchError } from 'rxjs/operators';
 import { environment } from 'src/environments/environment'; // Ajustar ruta si es necesario
 import { Vehiculo } from '../models/vehiculo.model'; // Ajustar ruta al modelo

 crear(vehiculoData: FormData): Observable<any> { // Usar FormData si se envían archivos
   // El token JWT es añadido por el AuthInterceptor.
    // No es necesario establecerlo manualmente aquí si el interceptor está activo para esta ruta.
                    return this.http.post<any>(`${environment.apiUrl}/vehiculos`, vehiculoData).pipe(
     catchError(error => {
       console.error('Error detallado al crear vehículo:', error);
       if (error.error && error.error.message) {
         console.error('Mensaje de error del servidor:', error.error.message);
       }
```

```
    // Aquí se podría usar ngx-toastr o un servicio de manejo de errores
        return throwError(() => new Error(error.error.message || 'Error al crear el vehículo'));
    })
  );
}
```

(Nota: El ejemplo original para crear un vehículo mostraba la adición manual de la cabecera Authorization. Si el AuthInterceptor está correctamente configurado y activo, este paso es generalmente redundante en el servicio. El ejemplo anterior asume que el interceptor maneja el token).

---

6. Base de Datos

6.1. Esquema de la Base de Datos

La base de datos de SWAPLT se compone de varias tablas diseñadas para almacenar la información de manera organizada y relacional. A continuación, se describen las tablas más importantes y sus campos clave:

- **Tablas Core (Entidades Principales)**
    - **users:**
        - **Propósito: Almacena la información de los usuarios registrados en la plataforma, gestionando la autenticación, roles y datos personales.**
        - **Campos Importantes Destacados: id (PK), name, email (único, para login), password, rol ('admin' o 'user'), avatar (URL de la imagen de perfil), verification_code (para verificación de email).**
    - **vehiculos:**
        - **Propósito: Contiene todos los detalles de los vehículos publicados por los usuarios, siendo el núcleo de la plataforma de compra-venta.**
        - **Campos Importantes Destacados: id (PK), user_id (FK a users), categoria_id (FK a categorias), marca, modelo, precio (decimal), anio, estado ('nuevo', 'usado'), transmision, tipo_combustible, kilometraje, descripcion, matricula.**
    - **categorias:**
        - **Propósito: Clasifica los vehículos en diferentes categorías para facilitar la búsqueda y organización.**
        - **Campos Importantes Destacados: id (PK), nombre (único).**
    - ---
- **Tablas de Relación y Características Adicionales**

- **vehiculos_imagenes:**
  - **Propósito: Almacena las rutas o URLs de las imágenes asociadas a cada vehículo.**
  - **Relaciones Clave: vehiculo_id (FK a vehiculos).**
  - **Campos Destacados: imagen_url, imagen_path, imagen_order (para ordenar las imágenes de un vehículo).**
- **mensajes:**
  - **Propósito: Gestiona la comunicación privada entre usuarios.**
  - **Relaciones Clave: emisor_id (FK a users), receptor_id (FK a users).**
  - **Campos Destacados: contenido (texto del mensaje), leido (estado de lectura).**
- **favoritos:**
  - **Propósito: Tabla pivote que permite a los usuarios marcar vehículos como favoritos.**
  - **Relaciones Clave: user_id (FK a users), vehiculo_id (FK a vehiculos).**
- **valoraciones:**
  - **Propósito: Permite a los usuarios calificar y comentar sobre otros usuarios con los que han interactuado.**
  - **Relaciones Clave: emisor_id (FK a users, quien valora), receptor_id (FK a users, quien es valorado).**
  - **Campos Destacados: valor (puntuación numérica), comentario.**
- **user_blocks:**
  - **Propósito: Tabla pivote para gestionar los bloqueos entre usuarios.**
  - **Relaciones Clave: blocker_id (FK a users, quien realiza el bloqueo), blocked_id (FK a users, quien es bloqueado).**
  - **Campos Destacados: razon (motivo del bloqueo).**
- **vehiculo_visitas:**

        - **Propósito: Registra las visitas que reciben los anuncios de vehículos.**
        - **Relaciones Clave: vehiculo_id (FK a vehiculos), user_id (FK a users, opcional, si el visitante está logueado).**
        - **Campos Destacados: ip_address, fecha_visita.**
    - **vehiculos_reportes:**
        - **Propósito: Almacena información sobre reportes generados para vehículos (ej. PDFs).**
        - **Relaciones Clave: vehiculo_id (FK a vehiculos).**
        - **Campos Destacados: pdf_url.**
- **Tablas del Sistema Laravel**

  Adicionalmente, el sistema utiliza tablas estándar de Laravel para funcionalidades internas:

    - **cache y cache_locks: Para el sistema de caché de Laravel.**
    - **jobs, job_batches, failed_jobs: Para el sistema de colas de Laravel.**
- **Notas Generales sobre el Esquema:**

    - **Todas las tablas principales incluyen campos created_at y updated_at para el seguimiento de registros (timestamps).**
    - **Se utilizan claves foráneas para mantener la integridad referencial entre las tablas.**
    - **Donde es apropiado, se han configurado eliminaciones en cascada (ON DELETE CASCADE) para manejar la consistencia de los datos al eliminar registros relacionados.**

6.2. Relaciones Importantes en la Base de Datos

Las tablas de la base de datos de SWAPLT están interconectadas a través de relaciones bien definidas, utilizando claves primarias y foráneas para asegurar la integridad referencial y la coherencia de los datos. Estas relaciones son la representación a nivel de base de datos de las asociaciones lógicas entre las diferentes entidades del sistema (usuarios, vehículos, mensajes, etc.).

https://drive.google.com/drive/u/0/folders/1hFgEu3ry18t5IfudMaAsWtKFtns53uca

- **La relación entre users y vehiculos (un usuario posee múltiples vehículos).**
- **La conexión entre vehiculos y vehiculo_imagenes (un vehículo tiene múltiples imágenes).**
- **Las relaciones que facilitan la mensajería entre users a través de la tabla mensajes.**
- **Las relaciones de muchos a muchos gestionadas a través de tablas pivote como favoritos (entre users y vehiculos) y user_blocks (entre users).**

6.3. Migraciones de Base de Datos

La gestión de la estructura de la base de datos (esquema) en SWAPLT se realiza mediante el sistema de migraciones de Laravel. Este enfoque permite definir y versionar los cambios en la base de datos de manera programática, facilitando la colaboración entre desarrolladores y la consistencia del esquema en diferentes entornos. Los archivos de migración se encuentran en el directorio `/database/migrations/` del proyecto backend.

- **Proceso Básico de Migraciones**

  El flujo de trabajo estándar con las migraciones en SWAPLT es el siguiente:

- **Creación de Nuevas Migraciones: Para generar un nuevo archivo de migración, se utiliza el comando Artisan:**

php artisan make:migration create_vehiculos_table

-
  - **Aplicación de Migraciones Pendientes: Para ejecutar todas las migraciones que aún no se han aplicado a la base de datos, se utiliza:**

php artisan migrate

-
  - **Este comando también se utiliza durante la configuración inicial del proyecto después de clonar el repositorio.**
  - **Reversión de Migraciones: Laravel permite revertir migraciones si es necesario:**

php artisan migrate:rollback

-
  - **Para revertir todas las migraciones aplicadas**

php artisan migrate:reset

-
  - **Para revertir todas las migraciones y luego volver a ejecutarlas todas (útil para reconstruir la base de datos en desarrollo):**

php artisan migrate:refresh

-
- **Seeders y Factories para Datos de Prueba**

Para poblar la base de datos con datos iniciales o de prueba, especialmente en entornos de desarrollo, SWAPLT utiliza Seeders y Factories de Laravel.

- **Seeders (/database/seeders/): Contienen la lógica para insertar datos en las tablas.**

    - **Para crear un nuevo seeder**

php artisan male:seeder VehiculosSeeder

-

    - **Para ejecutar todos los seeders definidos (generalmente invocando al DatabaseSeeder principal):**

php artisan db:seed

-

    - **Para ejecutar un seeder específico**

php artisan db:seed --class=VehiculosSeeder

-
- **Factories (/database/factories/): Definen "plantillas" para generar instancias de modelos Eloquent con datos falsos (usando la librería Faker), útiles para crear grandes cantidades de datos de prueba de manera dinámica para los seeders o pruebas automatizadas.**

- **Consideraciones Específicas y Buenas Prácticas en SWAPLT**

  Al trabajar con migraciones en el proyecto SWAPLT, se deben tener en cuenta las siguientes convenciones y prácticas:

    - **Convenciones de Nombrado para Archivos de Migración:**

        - **Los archivos siguen el formato estándar de Laravel: YYYY_MM_DD_HHMMSS_nombre_descriptivo.php.**
        - **Se utilizan nombres descriptivos para indicar el propósito (ej. create_users_table, add_rol_to_users_table).**
    - **Buenas Prácticas:**

        - **Orden de las Migraciones: Las migraciones se ejecutan cronológicamente según el timestamp en su nombre. Es crucial considerar las dependencias entre tablas; por ejemplo, una tabla con una clave foránea debe crearse después de la tabla a la que referencia.**

Manejo de Claves Foráneas: Se utiliza el método foreignId() de Laravel para definir claves foráneas, y se especifican las acciones onDelete() (ej. cascade) y onUpdate() según la lógica de negocio.

```
// Ejemplo de definición de clave foránea en una migración

$table->foreignId('user_id')

    ->constrained() // Asume que la tabla es 'users' y la columna 'id'

    ->onDelete('cascade');
```

    -
        - **Modificaciones de Tablas Existentes: Para modificar tablas ya creadas, se utiliza Schema::table() en el método up() de la migración. Es recomendable verificar la existencia de columnas antes de intentar modificarlas o eliminarlas.**
        - **Advertencias Importantes:**

        - **Backups: Siempre realizar un backup de la base de datos antes de ejecutar migraciones en un entorno de producción.**
        - **Migraciones en Producción: Evitar modificar migraciones que ya se han ejecutado en producción. En su lugar, crear nuevas migraciones para aplicar los cambios necesarios.**
        - **Rendimiento: Las migraciones que alteran tablas muy grandes pueden tardar tiempo en ejecutarse y deben planificarse para horarios de bajo tráfico. Considerar el uso de índices para mejorar el rendimiento de las tablas.**
    - **Migraciones Complejas en SWAPLT:**
        - **Se debe prestar especial atención a las migraciones que involucran la tabla de usuarios y la gestión de roles debido a sus relaciones.**
        - **Las migraciones que afectan a datos transaccionales o financieros (si aplicara) deben manejar la integridad referencial con sumo cuidado.**
        - **Las migraciones para la importación o transformación de datos históricos deben incluir validaciones y un adecuado manejo de errores.**
- **Comandos Artisan Útiles para Migraciones**

Además de los comandos básicos, los siguientes comandos de Artisan son útiles para gestionar y diagnosticar migraciones:

Ver el estado de todas las migraciones (cuáles se han ejecutado y cuáles están pendientes):

php artisan migrate:status

- **(Nota: php artisan migrate:show no es un comando estándar de Laravel para ver la estructura de una tabla. Para inspeccionar la estructura, se suelen usar herramientas de base de datos o comandos SQL directos).**

Forzar la ejecución de migraciones en producción (usar con precaución, ya que omite las confirmaciones):

php artisan migrate --force

Este sistema de migraciones asegura que la estructura de la base de datos de SWAPLT evolucione de manera controlada y reproducible.

---

7. Pruebas

Para asegurar la calidad, fiabilidad y mantenibilidad de la aplicación SWAPLT, se ha implementado una estrategia de pruebas que combina pruebas automatizadas tanto en el backend como el frontend, además de pruebas manuales para la API.

7.1. Estrategia General de Pruebas

El enfoque de pruebas en SWAPLT busca cubrir diferentes niveles:

- **Pruebas Automatizadas del Backend: Se centran en verificar la lógica de negocio, la integridad de los datos y el correcto funcionamiento de la API de Laravel.**
- **Pruebas Automatizadas del Frontend: Aseguran la correcta renderización y comportamiento de los componentes de Angular, la lógica de los servicios del frontend y los flujos de usuario principales a través de pruebas End-to-End.**
- **Pruebas Manuales de API: Utilizadas durante el desarrollo y para verificaciones puntuales de los endpoints del backend.**

Tanto las pruebas automatizadas del backend como las del frontend están diseñadas para ser integradas en un pipeline de CI/CD (Integración Continua/Despliegue Continuo), permitiendo la ejecución automática de las mismas ante cambios en el código.

7.2. Pruebas del Backend (Laravel)

El testing en el backend de Laravel se realiza utilizando PHPUnit como framework principal.

- **Tipos de Pruebas Implementadas:**
    - **Pruebas Unitarias (Unit Tests):**
        - **Ubicación: Directorio tests/Unit/.**

        - **Enfoque: Pruebas de componentes individuales y aislados del resto de la aplicación, como la validación de modelos Eloquent, la lógica de negocio encapsulada en servicios, métodos de utilidad y transformaciones de datos.**
    - **Pruebas de Características/Integración (Feature Tests):**
        - **Ubicación: Directorio tests/Feature/.**
        - **Enfoque: Pruebas de funcionalidades completas desde la perspectiva de una petición HTTP a la API, verificando la respuesta, las interacciones con la base de datos, la autenticación y autorización, y los flujos de negocio completos.**
- **Herramientas Adicionales Utilizadas:**
    - **FakerPHP: Para la generación de datos de prueba falsos, utilizado principalmente a través de las factories de Laravel.**
    - **Mockery: Para la creación de objetos mock (dobles de prueba) cuando es necesario aislar el código bajo prueba de sus dependencias.**
- **Ejecución de las Pruebas del Backend: Desde la carpeta raíz del proyecto backend, se pueden utilizar los siguientes comandos:**
    - **Para ejecutar todas las pruebas (recomendado):**

php artisan test

-
    - **Alternativamente (ejecutando PHPUnit directamente):**

./vendor/bin/phpunit

-

Se prefiere php artisan test ya que utiliza la configuración específica de Laravel para el entorno de pruebas (definido en .env.testing) y ofrece una salida más amigable.

7.3. Pruebas del Frontend (Angular)

El testing en el frontend de Angular se aborda con pruebas unitarias y pruebas End-to-End (E2E).

- **Pruebas Unitarias:**
  - **Herramientas: Se utiliza Jasmine como framework de pruebas y Karma como corredor de pruebas, que son las herramientas estándar en Angular. También se emplea el Angular Testing Module para facilitar la simulación y gestión de dependencias en los tests.**
  - **Enfoque:**
    - **Componentes: Pruebas de renderizado, interacción y comportamiento de componentes clave como formularios (registro, login), gestión de perfiles, visualización de datos y componentes de navegación.**
    - **Servicios: Pruebas de la lógica de negocio en servicios como los de autenticación, gestión de usuarios, comunicación con el backend y manejo de estado.**
  - **Ubicación: Los archivos de prueba unitaria (.spec.ts) se encuentran generalmente junto a los archivos del componente o servicio que prueban.**
  - **Ejecución: Desde la carpeta raíz del proyecto frontend:**
    - **Ejecutar todas las pruebas unitarias: ng test**
    - **Ejecutar pruebas y generar informe de cobertura: ng test --code-coverage**
    - **Ejecutar pruebas en modo "watch" (se re-ejecutan al detectar cambios): ng test --watch**

        - **Ejecutar pruebas específicas por patrón de nombre: ng test --include=**/nombre.service.spec.ts**
- **Pruebas End-to-End (E2E):**
    - **Herramienta: Se utiliza Cypress para las pruebas E2E, ofreciendo una alternativa moderna a Protractor.**
    - **Enfoque: Cubren flujos completos de usuario simulando la interacción real, incluyendo la integración con el backend, validación de formularios, navegación entre páginas y manejo de estados de autenticación.**
    - **Ubicación: Los archivos de prueba E2E de Cypress se encuentran típicamente en una carpeta cypress/e2e/.**
    - **Ejecución: Desde la carpeta raíz del proyecto frontend:**
        - **Abrir la interfaz gráfica de Cypress: npx cypress open**
        - **Ejecutar todas las pruebas E2E en modo headless: npx cypress run**
        - **Ejecutar en un navegador específico: npx cypress run --browser chrome**
        - **Ejecutar un archivo de prueba E2E específico: npx cypress run --spec "cypress/e2e/auth/login.cy.ts"**
    - **Herramientas Adicionales para E2E: Se utiliza Mock Service Worker (MSW) para simular respuestas del backend durante las pruebas E2E, permitiendo tests más controlados y fiables.**
- **Cobertura de Pruebas (Frontend):**
    - **Se mantuvo una cobertura de pruebas unitarias de aproximadamente 70-80% para los componentes y servicios críticos.**
    - **Las pruebas E2E se enfocaron en los flujos principales de la aplicación.**
- **Buenas Prácticas Implementadas (Frontend):**
    - **Uso del patrón Page Objects para organizar el código de las pruebas E2E.**

- Implementación de mocks para servicios externos o dependencias complejas.
- Diseño de pruebas aisladas y deterministas.
- Documentación de los casos de prueba.

7.4. Pruebas Manuales de API

Además de las pruebas automatizadas, se realizaron pruebas manuales de la API del backend durante la fase de desarrollo y para verificaciones específicas.

- Herramienta: Postman.
- Recursos: Se dispone de una colección de Postman (TFC.postman_collection.json) que contiene un conjunto de peticiones predefinidas para los diferentes endpoints de la API. Esta colección puede ser importada en Postman para facilitar la ejecución de pruebas manuales, la exploración de la API y la depuración. Se recomienda a los desarrolladores utilizar y actualizar esta colección.

Esta combinación de pruebas automatizadas y manuales contribuye a la robustez y calidad general del proyecto SWAPLT.

8. Contribución y Mantenimiento

Esta sección establece las directrices y convenciones para asegurar que el proyecto SWAPLT sea mantenible a lo largo del tiempo y para facilitar la colaboración si otros desarrolladores se unen al proyecto o si tú mismo, D10S, retomas el desarrollo en el futuro.

8.1. Guia de Estilo de Codigo

Mantener un estilo de código consistente es crucial para la legibilidad y el mantenimiento del proyecto.

- **Backend (PHP/Laravel)**

  Para el desarrollo del backend con PHP y Laravel en SWAPLT, se siguen las siguientes pautas y convenciones, inspiradas principalmente en PSR-12 y las prácticas estándar de Laravel:

  - **Estándares Generales de Código (basados en PSR-12):**

    - **Indentación: 4 espacios (no tabulaciones).**
    - **Longitud de Línea: Se recomienda un máximo de 120 caracteres por línea.**
    - **Espaciado: Consistente alrededor de operadores, después de comas, y antes y después de llaves.**

        - **Estructuras de Control: Uso de llaves ({}) para todas las estructuras de control (if, else, for, foreach, while, switch), incluso si el cuerpo es de una sola línea.**
        - **Nomenclatura:**
            - **Clases: PascalCase (ej. VehiculoController).**
            - **Métodos y Variables: camelCase (ej. obtenerVehiculos, $listaVehiculos).**
            - **Constantes: UPPER_CASE con guiones bajos para separar palabras (ej. MAX_INTENTOS).**
    - **Convenciones Específicas de Laravel:**
        - **Modelos: Nombres en singular y PascalCase (ej. User, Vehiculo).**
        - **Tablas de Base de Datos: Nombres en plural y snake_case (ej. users, vehiculo_imagenes).**
        - **Controladores: Nombres con el sufijo Controller y PascalCase (ej. AuthController).**
        - **Rutas Nombradas: Se recomienda kebab-case (ej. user-profile, vehiculos.index).**
        - **Archivos de Migración: Timestamp seguido de una descripción en snake_case (ej. 2025_06_04_000000_create_vehiculos_table).**
        - **Seeders: Nombres con el sufijo Seeder y PascalCase (ej. VehiculoSeeder).**
    - **Documentación y Comentarios:**
        - **DocBlocks: Utilizar DocBlocks (bloques de documentación PHPDoc) para clases, métodos y funciones importantes, detallando su propósito, parámetros (@param), valor de retorno (@return) y excepciones que puede lanzar (@throws).**

    - **Comentarios en Línea: Usar // para comentarios de una sola línea que expliquen lógica compleja o decisiones de diseño no obvias. Deben ser concisos y relevantes.**
  - **Documentación de API: Se menciona la intención de usar anotaciones de OpenAPI/Swagger para documentar los endpoints de la API, incluyendo ejemplos de peticiones/respuestas y códigos de error.**

  - **Buenas Prácticas de Organización de Código:**

    - **Controladores Delgados: La lógica de negocio compleja no debe residir en los controladores.**
    - **Servicios: Mover la lógica de negocio a clases de servicio dedicadas (en app/Services/).**
    - **Repositorios (Opcional): Para lógica de acceso a datos más compleja o para desacoplar los controladores/servicios de Eloquent, se puede usar el patrón repositorio (en app/Repositories/).**
    - **Requests Personalizadas: Para la validación de las peticiones HTTP (en app/Http/Requests/).**
    - **Excepciones Personalizadas: Para un manejo de errores más granular (en app/Exceptions/).**
    - **Nombres Descriptivos: Utilizar nombres claros y descriptivos para variables, métodos y clases.**
  - **Herramientas Recomendadas para el Estilo y Análisis (Backend):**

    - **Laravel Pint: Para formateo automático del código (se recomienda su uso).**
    - **PHP_CodeSniffer: Para análisis estático y detección de violaciones de estándares de código.**

    - **PHPStan: Para análisis de tipos y detección de errores antes de la ejecución.**
    - **Laravel IDE Helper: Para mejorar el autocompletado en el IDE.**

8.2. Flujo de Trabajo con Git

El control de versiones del proyecto SWAPLT se realiza con Git, y el repositorio está alojado en GitHub.

- **Estrategia de Ramas: Para este proyecto, el desarrollo se ha realizado principalmente directamente sobre la rama main. Si bien esto fue una decisión tomada para simplificar el flujo en un proyecto individual, para futuros desarrollos o colaboraciones se recomendaría adoptar una estrategia de ramificación más robusta, como GitFlow o el uso de ramas de características (feature branches) para cada nueva funcionalidad o corrección, fusionándolas a main a través de Pull Requests.**
- **Convenciones para Mensajes de Commit: Los mensajes de commit realizados durante el desarrollo de SWAPLT han buscado ser descriptivos de los cambios implementados en cada tramo. Para futuras contribuciones, se podría considerar la adopción de un estándar como "Conventional Commits" para una mayor claridad y la posibilidad de generar automáticamente registros de cambios (changelogs). Ejemplo de un mensaje de commit descriptivo: feat: Implementa paginación en listado de vehículos.**

---

9. Glosario

Este glosario define los términos técnicos, acrónimos y conceptos específicos del proyecto SWAPLT utilizados a lo largo de este manual, con un enfoque particular en el backend desarrollado con Laravel. Su objetivo es facilitar la comprensión y asegurar una interpretación homogénea de la terminología clave.

9.1. Terminología Específica de SWAPLT

- **Modelos y Entidades Principales**
  - **User (Usuario)**
    - **Definición: Modelo principal que representa a los usuarios de la plataforma SWAPLT. Implementa autenticación JWT y maneja roles de usuario (ej. 'admin', 'user').**
    - **Características Clave: Gestión de bloqueos entre usuarios, valoraciones, e históricamente verificación de email.**
  - **Vehiculo (Vehículo)**
    - **Definición: Modelo central que representa los vehículos en venta o intercambio en la plataforma. Contiene toda la información técnica y comercial del vehículo.**
    - **Características Clave: Mantiene relaciones con categorías, imágenes del vehículo, reportes (si aplica), mensajes relacionados, favoritos y visitas.**
  - **Categoria (Categoría)**
    - **Definición: Modelo que clasifica los vehículos en diferentes categorías (ej. coches, motos, furgonetas) para facilitar su organización y la búsqueda por parte de los usuarios.**
    - **Características Clave: Establece una relación de uno a muchos con los vehículos.**
  - **Mensaje**
    - **Definición: Modelo que gestiona la comunicación directa y privada entre usuarios a través del sistema de mensajería interna de la plataforma.**

        - **Características Clave: Mantiene relaciones con un usuario emisor y un usuario receptor; su funcionalidad está integrada con el sistema de bloqueo entre usuarios.**
    - **Valoracion (Valoración)**
        - **Definición: Modelo que maneja las valoraciones (puntuaciones y comentarios) que los usuarios pueden otorgarse mutuamente después de una interacción o transacción.**
        - **Características Clave: Incluye un sistema de puntuación (ej. 1-5), comentarios opcionales y, potencialmente, validación de interacción previa entre los usuarios involucrados.**
    - **UserBlock (Bloqueo de Usuario)**
        - **Definición: Modelo que gestiona los bloqueos entre usuarios, permitiendo a un usuario prevenir interacciones no deseadas con otro.**
        - **Características Clave: Registra quién bloquea a quién y puede incluir la razón del bloqueo. Las relaciones son bidireccionales en términos de consulta (quién me ha bloqueado y a quién he bloqueado).**
    - **VehiculoImagen (Imagen de Vehículo)**
        - **Definición: Modelo que maneja las imágenes asociadas a cada vehículo listado en la plataforma, incluyendo su orden de visualización y la información para su almacenamiento y acceso.**
        - **Características Clave: Permite la gestión de múltiples imágenes por vehículo con un orden específico.**
- **Controladores y Lógica de Negocio Clave**
    - **AuthController**
        - **Definición: Controlador de Laravel responsable de gestionar todos los aspectos de la autenticación y el registro de usuarios en la API.**

        - **Características Clave: Incluye la lógica para el registro de nuevas cuentas, la verificación de correo electrónico (si aplica), el inicio de sesión mediante credenciales (email y password) generando un token JWT, y la funcionalidad de recuperación de contraseña.**
    - **VehiculoController**
        - **Definición: Controlador principal para la gestión integral de los vehículos dentro de la aplicación.**
        - **Características Clave: Maneja las operaciones CRUD (Crear, Leer, Actualizar, Borrar) para los vehículos, así como la lógica para la búsqueda avanzada, la aplicación de filtros y la gestión de las imágenes asociadas a cada vehículo.**
    - **MensajeController**
        - **Definición: Controlador que gestiona la funcionalidad del sistema de mensajería interna entre los usuarios de la plataforma.**
        - **Características Clave: Encargado del envío de mensajes, la verificación de bloqueos entre usuarios antes de permitir la comunicación y la obtención del historial de conversaciones.**
    - **ValoracionController**
        - **Definición: Controlador que maneja la lógica del sistema de valoraciones entre usuarios.**
        - **Características Clave: Permite la creación de nuevas valoraciones, incluyendo validaciones como la de interacción previa entre los usuarios involucrados.**
    - **CategoriaController**
        - **Definición: Controlador destinado a la gestión de las categorías de vehículos.**
        - **Características Clave: Proporciona operaciones CRUD para las categorías, con posibles restricciones de acceso para roles de administrador.**

- **Middleware y Seguridad**
    - **CorsMiddleware**
        - **Definición: Middleware personalizado de Laravel que gestiona las políticas de Cross-Origin Resource Sharing (CORS) para la API de SWAPLT.**
        - **Características Clave: Configura las cabeceras HTTP necesarias para permitir o restringir las solicitudes provenientes de diferentes dominios (como el del frontend Angular) al backend.**

9.2. Conceptos Técnicos Fundamentales

- **JWT (JSON Web Token)**
    - **Definición: Estándar abierto (RFC 7519) basado en JSON para crear tokens de acceso que permiten la propagación segura de identidad y privilegios. En SWAPLT, se utiliza para la autenticación stateless de la API.**
    - **Uso Específico en SWAPLT: Implementado a través de la librería tymon/jwt-auth en Laravel para generar, validar y gestionar los tokens que los clientes (como el frontend Angular) usan para autenticar sus peticiones.**
- **Eloquent ORM**
    - **Definición: Es el mapeador objeto-relacional (ORM) incluido con Laravel, que proporciona una implementación Active Record para trabajar con la base de datos de forma intuitiva.**
    - **Uso Específico en SWAPLT: Utilizado para definir los Modelos (como User, Vehiculo, etc.), gestionar las relaciones entre ellos (ej. hasMany, belongsTo), definir scopes para consultas reutilizables y manejar eventos del ciclo de vida de los modelos.**
- **API REST (Representational State Transfer)**

- **Definición: Estilo de arquitectura para diseñar aplicaciones en red, basado en el protocolo HTTP. SWAPLT expone su funcionalidad del backend a través de una API REST.**
- **Características en SWAPLT: Los endpoints de la API devuelven respuestas en formato JSON, utilizan los métodos HTTP estándar (GET, POST, PUT, DELETE), se basan en URLs para identificar recursos y emplean autenticación JWT para las rutas protegidas, con respuestas estandarizadas para éxitos y errores.**

9.3. Patrones de Diseño Aplicados

- **MVC (Model-View-Controller)**
  - **Definición: Patrón de arquitectura de software que separa la lógica de negocio y la representación de los datos de la interfaz de usuario.**
  - **Implementación en SWAPLT (Laravel): Laravel sigue intrínsecamente este patrón, donde los Modelos gestionan los datos (interacción con la base de datos vía Eloquent), los Controladores manejan la lógica de las peticiones del usuario y las Vistas (aunque en una API REST se refieren más a la transformación de datos a JSON) presentan la información. Las rutas de la API conectan las peticiones a las acciones de los controladores.**
- **Repository Pattern (Patrón Repositorio)**
  - **Definición: Patrón de diseño que media entre el dominio y las capas de mapeo de datos usando una interfaz similar a una colección para el acceso a los objetos de dominio.**
  - **Uso en SWAPLT (mencionado como buena práctica): Aunque no se ha detallado su implementación extensiva, se considera una buena práctica para abstraer la lógica de acceso a datos, moviéndola de los controladores o servicios a clases de repositorio dedicadas, especialmente para consultas complejas o para facilitar el testing.**

9.4. Acrónimos y Abreviaturas Comunes

- **CRUD**
  - **Definición: Acrónimo de las operaciones básicas en la persistencia de datos: Create (Crear), Read (Leer), Update (Actualizar), Delete (Borrar).**
  - **Uso en SWAPLT: Estas operaciones son fundamentales y están implementadas en la mayoría de los controladores principales para la gestión de recursos como usuarios, vehículos, categorías, etc.**
- **API**
  - **Definición: Application Programming Interface (Interfaz de Programación de Aplicaciones).**
  - **Uso en SWAPLT: Se refiere a la interfaz de comunicación del backend de Laravel, que expone los endpoints RESTful para la interacción con el frontend u otros clientes.**
- **ORM**
  - **Definición: Object-Relational Mapping (Mapeo Objeto-Relacional).**
  - **Uso en SWAPLT: Implementado a través de Eloquent para facilitar la interacción entre el código PHP orientado a objetos y la base de datos relacional MySQL.**

9.5. Términos Clave del Dominio de SWAPLT

- **Valoración (Usuario)**
  - **Definición: Sistema que permite a los usuarios de SWAPLT otorgar una puntuación (ej. de 1 a 5 estrellas) y un comentario a otros usuarios con los que han interactuado.**
  - **Contexto en SWAPLT: Esta funcionalidad está diseñada para construir confianza en la plataforma y generalmente está disponible después de que se ha establecido algún tipo de interacción previa (ej. a través del sistema de mensajería).**

- **Bloqueo (Usuario)**
    - **Definición: Mecanismo de seguridad y preferencia que permite a un usuario prevenir cualquier tipo de interacción (como visualización de anuncios o envío de mensajes) con otro usuario específico en la plataforma.**
    - **Contexto en SWAPLT: Implementado a través del modelo UserBlock y sus relaciones, afectando la visibilidad de contenidos y la capacidad de comunicación.**
- **Categoría (Vehículo)**
    - **Definición: Clasificación utilizada para organizar los vehículos listados en SWAPLT (ej. "Coches", "Motos", "Furgonetas", "Clásicos").**
    - **Contexto en SWAPLT: Ayuda a los usuarios a filtrar y encontrar vehículos de su interés de manera más eficiente. La gestión de categorías puede estar reservada a roles de administrador.**
- **Visita (Vehículo)**
    - **Definición: Registro de cada vez que un anuncio de vehículo es visualizado en detalle por un usuario (registrado o anónimo).**
    - **Contexto en SWAPLT: Se utiliza para el seguimiento de la popularidad de los anuncios y para generar estadísticas que pueden ser útiles para los vendedores.**
- **Favorito (Vehículo)**
    - **Definición: Funcionalidad que permite a los usuarios guardar o marcar vehículos de su interés para un acceso rápido y seguimiento posterior.**
    - **Contexto en SWAPLT: Implementado como una relación de muchos a muchos entre usuarios y vehículos.**